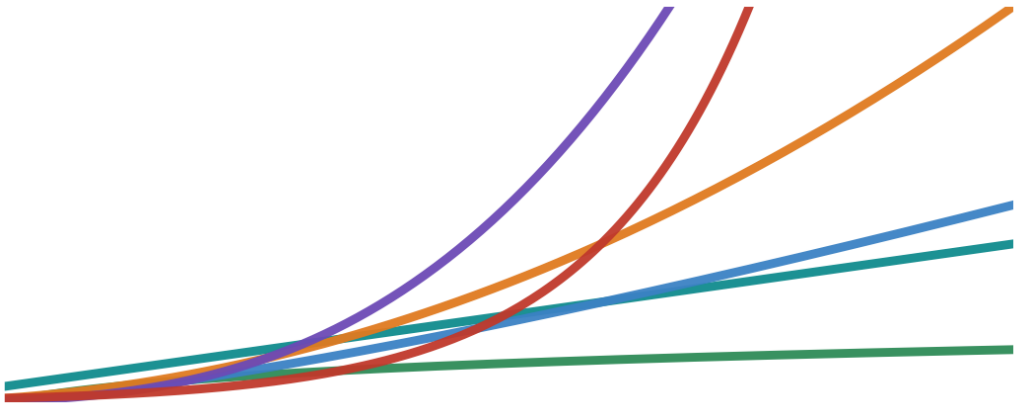
DSA WITH JAVA • COMPLEXITY ANALYSIS • INTERVIEW PREP

Time and Space Complexity in DSA with Java

Zero to Interview Level

Big-O • 8 growth classes • loops & recursion • searching & sorting
15 problems • 30 MCQs • 20 interview questions • cheat sheet • revision page



What you will learn

- Count the work of any loop or recursion, step by step, and simplify it to Big-O
- Tell time complexity from auxiliary space, including the recursion stack
- Analyse linear and binary search and six sorting algorithms, with traces and measured counts
- Solve 15 practice problems from brute force to optimised Java, then test yourself with 30 MCQs, 20 interview questions and 10 code-analysis exercises

Verified. Every Java listing was compiled and run on OpenJDK 21. Operation counts in the tables were measured by running instrumented code and then checked against closed-form formulas. Each optimised problem solution was cross-checked against its brute-force version on thousands of random inputs.

All 15 problems, 30 MCQs, 20 interview questions and 10 exercises are original practice material written for this guide. They are practice questions in the style of coding interviews; they are not claimed to be actual Google, Microsoft or LeetCode questions.

Contents

Welcome: How to Use This Guide 5

1. Why Complexity Matters 6

1.1 Two programs, the same answer, very different speed 6
1.2 Count steps, not seconds 6
1.3 What counts as one step? 6
1.4 What is n? 7
1.5 Growth rates at a glance 7
1.6 Eight analogies to carry with you 9

2. Big-O: The Language of Growth 10

2.1 The definition in plain English 10
2.2 The five simplification rules 10
2.3 Which term dominates? 10
2.4 A complete worked example 11
2.5 Big-O, Big-Omega, Big-Theta 11
2.6 Best, average and worst case 12
2.7 Common mistakes 12

3. The Eight Growth Classes, One by One 13

3.1 O(1) — constant time 13
3.2 O(log n) — logarithmic time 14
3.3 O(n) — linear time 16
3.4 O(n log n) — linearithmic time 17
3.5 O(n^2) — quadratic time 18
3.6 O(n^3) — cubic time 19
3.7 O(2^n) — exponential time 21
3.8 O(n!) — factorial time 23
3.9 Chapter summary 24

4. The Loop Analysis Toolbox 26

4.1 Single loops 26
4.2 Nested loops 26
4.3 Separate (sequential) loops 26
4.4 Constant-bound inner loops 27
4.5 Doubling and halving loops 27
4.6 Triangular loops 27
4.7 Three nested loops 28

4.8 Dependent loops . . . 29
4.9 Loops with early breaks . . . 31
4.10 Pattern summary . . . 32

5. Space Complexity . . . 33

5.1 Total space versus auxiliary space . . . 33
5.2 Variables and fixed-size structures: O(1) . . . 33
5.3 Arrays and matrices . . . 33
5.4 In-place algorithms . . . 34
5.5 The recursion stack . . . 35
5.6 Java-specific space notes . . . 36
5.7 The time–space trade-off . . . 36

6. Recursion and Recurrences . . . 37

6.1 $T(n) = T(n - 1) + c \rightarrow O(n)$ . . . 37
6.2 $T(n) = T(n / 2) + c \rightarrow O(\log n)$ . . . 37
6.3 $T(n) = 2T(n / 2) + n \rightarrow O(n \log n)$ . . . 37
6.4 $T(n) = 2T(n - 1) + c \rightarrow O(2^n)$ . . . 37
6.5 $T(n) = T(n - 1) + n \rightarrow O(n^2)$ . . . 37
6.6 $T(n) = 2T(n / 2) + c \rightarrow O(n)$ . . . 38
6.7 The Master Theorem (shortcut) . . . 38
6.8 Recursion and space . . . 38

7. Searching Algorithms . . . 39

7.1 Linear search . . . 39
7.2 Binary search . . . 39

8. Sorting Algorithms . . . 42

8.1 Bubble sort . . . 42
8.2 Selection sort . . . 42
8.3 Insertion sort . . . 43
8.4 Merge sort . . . 44
8.5 Quick sort . . . 45
8.6 Heap sort . . . 46
8.7 Comparison of all six . . . 47

9. Fifteen Practice Problems . . . 49

9.1 Problem 1: Pair With Target Sum . . . 49
9.2 Problem 2: First Non-Repeating Character . . . 50
9.3 Problem 3: Best Window Sum . . . 51
9.4 Problem 4: Sorted Squares . . . 52

9.5 Problem 5: Missing Number . . . 53
9.6 Problem 6: Merge Two Sorted Arrays . . . 54
9.7 Problem 7: Rotate Array Right by k . . . 55
9.8 Problem 8: Count Subarrays With Sum k . . . 56
9.9 Problem 9: Longest Substring Without Repeats . . . 57
9.10 Problem 10: Find a Peak . . . 59
9.11 Problem 11: Integer Square Root . . . 60
9.12 Problem 12: Stair Climb Ways . . . 61
9.13 Problem 13: Product Except Self . . . 61
9.14 Problem 14: Search a Staircase Matrix . . . 62
9.15 Problem 15: K-th Largest Element . . . 63

10. Thirty Multiple-Choice Questions . . . 65

10.1 Beginner level (questions 1–10) . . . 65
10.2 Intermediate level (questions 11–20) . . . 66
10.3 Interview level (questions 21–30) . . . 68

11. Twenty Interview Questions . . . 70

12. Ten Final Code-Analysis Exercises . . . 73

13. Cheat Sheet . . . 76

13.1 The growth ladder . . . 76
13.2 Loop patterns . . . 76
13.3 Searching and sorting . . . 76
13.4 Java collections . . . 77
13.5 Sums and series . . . 77
13.6 What n can I afford? . . . 78

14. One-Page Revision Sheet . . . 79

15. Answers and Explanations . . . 80

15.1 MCQ answer key . . . 80
15.2 MCQ explanations . . . 80
15.3 Solutions to the ten code-analysis exercises . . . 81
15.4 How this guide was verified . . . 82

Appendix: Maths Refresher . . . 84

Welcome: How to Use This Guide

This guide takes you from "what is Big-O?" to the complexity questions asked in internship and entry-level coding interviews. It assumes you can read basic Java: variables, `for` and `while` loops, arrays, methods and simple recursion. Nothing else.

The learning path

Roadmap

Part	Chapters	What you can do after it
Foundations	1 – 2	Explain why complexity matters and simplify any step count to Big-O.
The eight growth classes	3	Recognise O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(n^3)$, $O(2^n)$, O(n!) from code and from an analogy.
Loops, space, recursion	4 – 6	Analyse every common loop shape, count memory, and solve simple recurrences.
Algorithms	7 – 8	Analyse linear and binary search; bubble, selection, insertion, merge, quick and heap sort.
Practice	9 – 12	15 problems, 30 MCQs, 20 interview questions, 10 code-analysis exercises.
Quick reference	13 – 14	Cheat sheet and a one-page revision sheet.
Answers	15	MCQ explanations and full solutions to the exercises.

Reading tips

- Type every code listing and run it. Change `n`, predict the count, then check.
- Trace tables show the program variables after each step. Cover the answer column and fill it in yourself first.
- Coloured boxes: **Key idea** is the one thing to remember; **Derivation** shows algebra with no steps skipped; **Watch out** marks common mistakes; **Interview tip** shows how to say it aloud.
- When the book says "measured", the number came from running Java code (OpenJDK 21), not from a formula.

Symbols used

Symbol	Meaning
n	The input size (array length, string length, or the number given).
log n	Logarithm base 2 unless written otherwise. The base never changes the Big-O class.
⌊x⌋ and ⌈x⌉	Round down and round up. ⌊3.7⌋ = 3, ⌈3.2⌉ = 4.
Σ	Sum. Σ i for i = 1..n means 1 + 2 + … + n.
O, Ω, Θ	Upper bound, lower bound, tight bound (chapter 2).
Auxiliary space	Extra memory the algorithm allocates, not counting the input (chapter 5).

> **Note.** The practice material is original. The 15 problems are written in the style of coding-interview platforms, but none of them is claimed to be an actual Google, Microsoft or LeetCode question.

1. Why Complexity Matters

1.1 Two programs, the same answer, very different speed

Suppose you must check whether an array of `n` numbers contains a duplicate. Here are two correct programs.

Listing 1.1: two correct duplicate checks

```
static boolean hasDuplicateSlow(int[] a) {
    for (int i = 0; i < a.length; i++)
        for (int j = i + 1; j < a.length; j++)
            if (a[i] == a[j]) return true;      // compare every pair
    return false;
}

static boolean hasDuplicateFast(int[] a) {
    java.util.HashSet<Integer> seen = new java.util.HashSet<>();
    for (int x : a) {
        if (!seen.add(x)) return true;          // add() returns false if x was already present
    }
    return false;
}
```

For `n` = 100,000 and no duplicates, the first program compares 4,999,950,000 pairs. The second does about 100,000 insertions. At roughly 10^8 to 10^9 simple operations per second, that is the difference between several seconds (or more) and a few milliseconds. Complexity analysis is the skill of predicting this difference **before** running the code.

1.2 Count steps, not seconds

Wall-clock time depends on the computer, the Java version, whether the JVM has warmed up, and what else is running. It tells you nothing about how the program behaves when the input grows. So we count **basic steps** as a function of the input size `n`, and study how that count grows.

> **Key idea.** Time complexity is not "how many seconds". It is "how the number of steps grows as the input grows". Space complexity is the same idea for memory.

1.3 What counts as one step?

We use a simple model: each of the following takes one unit of time, whatever the size of the input.

Table 1.1: unit-cost operations

Counts as 1 step	Example in Java
Arithmetic on primitives	`x + y`, `n * 2`, `i % 3`
Comparison	`a[i] < a[j]`, `i == n`
Assignment	`sum = 0`; `a[i] = 5;`
Array element read or write	`a[i]` (the address is computed directly)
Method call overhead and return	`return x;`
Field or `length` access	`a.length`

Some innocent-looking calls are **not** single steps because they hide a loop inside:

Table 1.2: hidden costs

Looks like 1 step	Really costs
`s.substring(i, j)`	O(j − i): characters are copied
`s1.equals(s2)`	O(length): compares characters
`list.contains(x)`, `list.indexOf(x)`	O(n): scans the list
`list.add(0, x)`, `list.remove(0)` on an `ArrayList`	O(n): shifts every element
`s += "a"` inside a loop	O(length of s) each time, so O(n²) for n appends
`Arrays.sort(a)`	O(n log n)

1.4 What is n?

- For an array or a string, `n` is its length.
- For a matrix with `r` rows and `c` columns the input size is `r × c`; if it is square, `n × n`.
- When a method receives a number (for example `static int f(int n)` that loops `n` times) we use the number itself as `n`.
- With two inputs, keep two letters: an algorithm that scans array `a` then array `b` is O(a + b), not O(n).

1.5 Growth rates at a glance

The table below uses exact values (log base 2, rounded). Notice how the right-hand columns explode.

Table 1.3: how many steps each growth class needs

n	1	log n	n	n log n	n^2	n^3	2^n	n!
10	1	3.3	10	33	100	1,000	1,024	3,628,800
100	1	6.6	100	664	10^4	10^6	$\approx 1.26 \times 10^{30}$	$\approx 9.33 \times 10^{157}$
1,000	1	10.0	1,000	9,966	10^6	10^9	$\approx 1.07 \times 10^{301}$	$\approx 4.02 \times 10^{2567}$
10^6	1	19.9	10^6	19,931,569	10^{12}	10^{18}	too large	too large

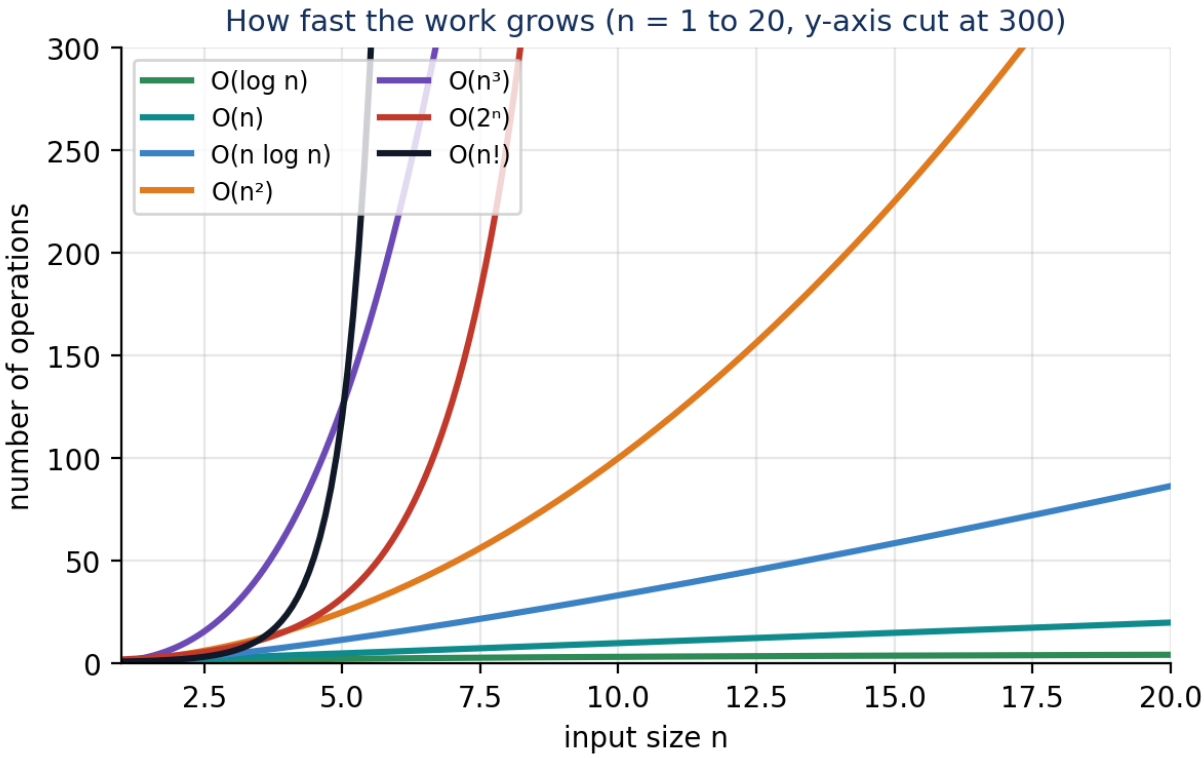


Figure 1. Small inputs hide the difference; by n = 20 the exponential and factorial curves have left the chart.

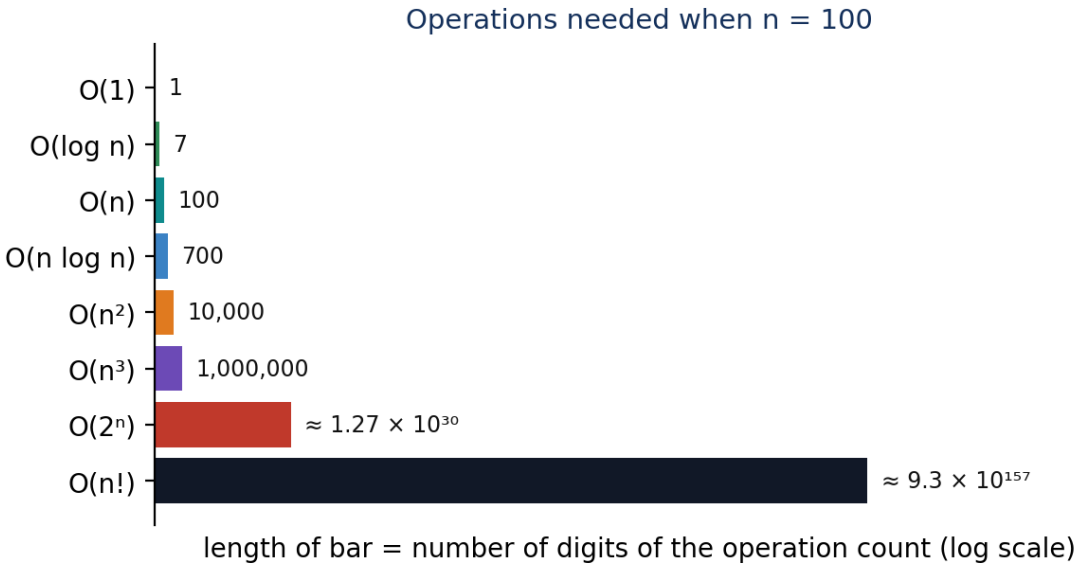


Figure 2. At n = 100, O(n³) is a million steps but O(2ⁿ) is already beyond anything a computer can do.

How big an n can I handle in one second?

Assume a computer performs about 10^8 simple operations per second (a common contest rule of thumb). The largest `n` for each class:

Table 1.4: input limits (10^8 operations)

Complexity	Largest n in about 1 second	Typical guideline
O(log n), O(1)	any practical n	—
O(n)	≈ 10^8	n up to 10^7 – 10^8
O(n log n)	≈ 4.5×10^6	n up to about 10^6
O(n^2)	≈ 10,000	n up to a few thousand
O(n^3)	≈ 464	n up to a few hundred
O(2^n)	≈ 26	n up to about 20 – 25
O(n!)	≈ 11	n up to about 10 – 11

Interview tip. In an interview, look at the constraint on n first. If $n \le 10^5$, an O(n^2) solution is too slow, and the interviewer expects O(n log n) or O(n). If $n \le 20$, they probably expect exponential backtracking.

1.6 Eight analogies to carry with you

Table 1.5: the eight classes and their analogies

Class	Everyday picture	Why it has that growth
O(1)	Take the top plate from a stack.	The size of the stack does not matter.
O(log n)	Find a word in a dictionary by opening near the middle and repeatedly halving.	Each step throws away half of what is left.
O(n)	Read every name on an attendance list.	One look per person.
O(n log n)	Sort a deck of cards by splitting into halves, sorting each, and merging.	log n rounds, and every round touches all n cards.
O(n^2)	Every guest at a party shakes hands with every other guest.	n guests × about n handshakes each.
O(n^3)	Check every possible trio of friends for a three-way conflict.	n choices for each of three positions.
O(2^n)	Packing a suitcase: each of n items is either in or out.	Every extra item doubles the number of choices.
O(n!)	Seat n people in a row in every possible order.	n choices for seat 1, n−1 for seat 2, and so on.

2. Big-O: The Language of Growth

2.1 The definition in plain English

We write **f(n) = O(g(n))** when, for large enough n, f(n) never grows faster than a constant multiple of g(n). Formally: there are constants $c > 0$ and n_0 such that $f(n) \le c \cdot g(n)$ for every $n \ge n_0$.

Derivation 2.1. Prove that 3n + 5 = O(n). We must find c and n_0. Try c = 4. We need $3n + 5 \le 4n$. Subtract 3n from both sides: $5 \le n$. So the inequality holds for every $n \ge 5$. Choose $n_0 = 5$. Check: n = 5 gives 3(5)+5 = 20 ≤ 4(5) = 20 ✓. Done.

Derivation 2.2. Prove that $2n^2 + 7n + 10 = O(n^2)$. Try c = 3. We need $2n^2 + 7n + 10 \le 3n^2$. Subtract $2n^2$: $7n + 10 \le n^2$. Test n = 8: 66 ≤ 64? No. Test n = 9: 73 ≤ 81? Yes. For $n \ge 9$ the right side n^2 grows faster than the left side 7n + 10 (the difference $n^2 - 7n - 10$ has increasing values from n = 4 onward), so it stays true. So c = 3 and $n_0 = 9$ work.

2.2 The five simplification rules

Table 2.1: simplification rules

Rule	What to do	Example
1. Drop constant factors	Multiplying by a constant never changes the class.	5n → n; n/2 → n; 100 → 1
2. Drop lower-order terms	Keep only the term that grows fastest.	$n^2 + 50n + 9 \to n^2$
3. Sequential code adds	Add the costs, then keep the larger.	O(n) then $O(n^2) \to O(n + n^2) = O(n^2)$
4. Nested code multiplies	Cost of the outer loop × cost of the body.	n iterations × O(log n) body → O(n log n)
5. Different inputs get different letters	Do not merge two sizes into one n.	Loop over a then over b: O(a + b); nested: O(a · b)

Key idea. The base of a logarithm never matters inside Big-O, because log_a n = log_b n / log_b a, and 1 / log_b a is just a constant. So $\log_2 n$, $\log_{10} n$ and ln n are all "O(log n)".

2.3 Which term dominates?

The order of common growth classes, from slowest to fastest:

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$

Table 2.2: simplification examples

Expression	Dominant term	Big-O	Why
$5n + 20$	$5n$	$O(n)$	n beats a constant; then drop the 5
$n^2 + 1000n$	n^2	$O(n^2)$	n^2 overtakes $1000n$ once $n > 1000$
$n + \log n$	n	$O(n)$	log n is far smaller than n
$n \log n + n$	$n \log n$	$O(n \log n)$	$n \log n \geq n$ whenever $\log n \geq 1$
$n^3 + n^2 \log n$	n^3	$O(n^3)$	$\log n < n$, so $n^2 \log n < n^3$
$2^n + n^{100}$	2^n	$O(2^n)$	every exponential eventually beats every polynomial
$3 \cdot 2^n + 100n^2$	$3 \cdot 2^n$	$O(2^n)$	drop the factor 3 and the polynomial
$n! + 2^n$	$n!$	$O(n!)$	factorial beats exponential

2.4 A complete worked example

We will count the work in this method line by line. Let $c_1, c_2, \ldots$ stand for the (unknown but constant) cost of one execution of each line.

Listing 2.1

```
static long sample(int[] a) {
    int n = a.length;
    long total = 0;
    for (int i = 0; i < n; i++)
        total += a[i];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            total += (long) a[i] * a[j];
    return total;
}
```

Table 2.3: line-by-line ledger

Line	Cost per run	Times it runs	Subtotal
`int n = a.length;`	c_1	1	c_1
`long total = 0;`	c_2	1	c_2
first loop: test, increment, `total += a[i]`	c_3	n (plus one final failing test)	$c_3 \cdot n + c_3$
second loop (outer)	c_4	n	$c_4 \cdot n$
inner loop body `total += a[i]*a[j]`	c_5	$n \times n = n^2$	$c_5 \cdot n^2$
`return total;`	c_6	1	c_6

Adding the subtotals: $T(n) = c_5 \cdot n^2 + (c_3 + c_4) \cdot n + (c_1 + c_2 + c_3 + c_6)$. Apply rule 2 (keep the dominant term) and rule 1 (drop its constant): **$T(n) = O(n^2)$**. The first loop is O(n), which is swallowed by the $O(n^2)$ nested loops. The auxiliary space is O(1): only `n`, `total`, `i`, `j`.

2.5 Big-O, Big-Omega, Big-Theta

Table 2.4: three bounds

Symbol	Meaning	Think of it as	Example
$O(g)$	Upper bound: grows no faster than g.	"at most"	Linear search is O(n).
$\Omega(g)$	Lower bound: grows at least as fast as g.	"at least"	Any comparison sort is $\Omega(n \log n)$.
$\Theta(g)$	Tight bound: both $O(g)$ and $\Omega(g)$.	"exactly this class"	Reading every element of an array is $\Theta(n)$.

In interviews people say "Big-O" and usually mean the tight bound. That is fine, but be precise when it matters: "binary search is O(log n) in the worst case" is correct; "binary search is O(n²)" is also technically true (an upper bound) but useless.

2.6 Best, average and worst case

Complexity can depend on *which* input of size `n` you get. We describe three situations:

- **Best case:** the luckiest input of size n.
- **Worst case:** the unluckiest input of size n. This is what Big-O quotes by default, because it is a guarantee.
- **Average case:** the expected cost over a stated distribution of inputs.

> **Derivation 2.3. Average case of linear search** (target is present, equally likely at each of n positions). Finding the element at position k costs k comparisons. Average = (1 + 2 + … + n) / n. Use the sum 1 + 2 + … + n = n(n+1)/2 (Gauss, proved in chapter 4). Then average = [n(n+1)/2] / n = (n+1)/2. This is about n/2 comparisons, which is O(n).

Table 2.5: cases for familiar algorithms

Algorithm	Best	Average	Worst	What triggers the worst case
Linear search	O(1)	O(n)	O(n)	target is last or absent
Binary search	O(1)	O(log n)	O(log n)	target is absent or found at the last probe
Insertion sort	O(n)	O(n²)	O(n²)	input sorted in reverse
Quick sort (last pivot)	O(n log n)	O(n log n)	O(n²)	input already sorted
Hash table lookup	O(1)	O(1)	O(n)	all keys land in one bucket (O(log n) in Java 8+ with tree bins)

2.7 Common mistakes

- **Counting lines instead of growth.** Ten lines inside a loop are still one constant-time body.
- **Forgetting hidden loops.** `contains`, `substring`, `String +=`, `ArrayList.remove(0)` (see Table 1.2).
- **Thinking O(2n) is bigger than O(n).** They are the same class.
- **Assuming a smaller Big-O is always faster.** For small n, an O(n²) loop with a tiny constant can beat an O(n log n) algorithm with a large constant.
- **Mixing up input size and input value.** A loop `for (i = 0; i < n; i++)` costs n steps where n is the *value* passed in. That is fine when n is the size, but the cost is exponential in the number of *digits* of n.
- **Ignoring the recursion stack** when counting space (chapter 5).

3. The Eight Growth Classes, One by One

Each section follows the same recipe: an analogy, Java code, measured operation counts, a trace table (the variables after every step), a time derivation, and auxiliary space. Chapter 4 then studies loop shapes in more detail.

3.1 O(1) — constant time

Analogy

Taking the top plate from a stack. It does not matter whether the stack has 5 plates or 5,000: one move.

Java code

Listing 3.1: constant-time methods

```
static int sumFirstN(int n) {
    return n * (n + 1) / 2;              // 1 multiply + 1 add + 1 divide = 3 steps, any n
}

static void swap(int[] a, int i, int j) {
    int tmp = a[i];                       // 1 read
    a[i] = a[j];                          // 1 read + 1 write
    a[j] = tmp;                           // 1 write  -> constant work, array size is irrelevant
}
```

Operation counts (measured)

The returned value changes with `n`, but the number of steps does not.

Table 3.1: O(1) means the step count is flat

n	sumFirstN(n) returned (measured)	Steps executed	Steps for swap
8	36	3	3
64	2,080	3	3
1024	524,800	3	3

Trace table: swap(a, 0, 2) on {4, 9, 7}

Table 3.2

Step	Statement	tmp	a[0]	a[2]	Array after the step
0	start	—	4	7	[4, 9, 7]
1	`int tmp = a[i];`	4	4	7	[4, 9, 7]
2	`a[i] = a[j];`	4	7	7	[7, 9, 7]
3	`a[j] = tmp;`	4	7	4	[7, 9, 4]

Time analysis

`swap` always executes exactly three assignments and a fixed number of array accesses. No statement depends on the length of `a`. So T(n) = c for some constant c, and **T(n) = O(1)**. For `sumFirstN` there are exactly three arithmetic operations, for any `n` (assuming the result fits in an `int`).

Auxiliary space

One extra `int` (`tmp`): **O(1)**.

> **Watch out.** O(1) does not mean "fast" or "one step". It means "does not grow with n". A method that always performs a million operations is still O(1).

3.2 O(log n) — logarithmic time

Analogy

Guess a secret number from 1 to 100. After each guess you hear "higher" or "lower", so you always guess the middle of what remains. At most 7 guesses are needed, because $2^7 = 128 \geq 100$.

Java code

Listing 3.2: halving and doubling loops

```
static int halving(int n) {
    int count = 0;
    while (n > 1) {
        n = n / 2;                      // the problem size is cut in half
        count++;
    }
    return count;                       // floor(log2 n) iterations
}
static int doubling(int n) {
    int count = 0;
    for (int i = 1; i < n; i = i * 2) {   // i = 1, 2, 4, 8, ...
        count++;
    }
    return count;                       // ceil(log2 n) iterations
}
```

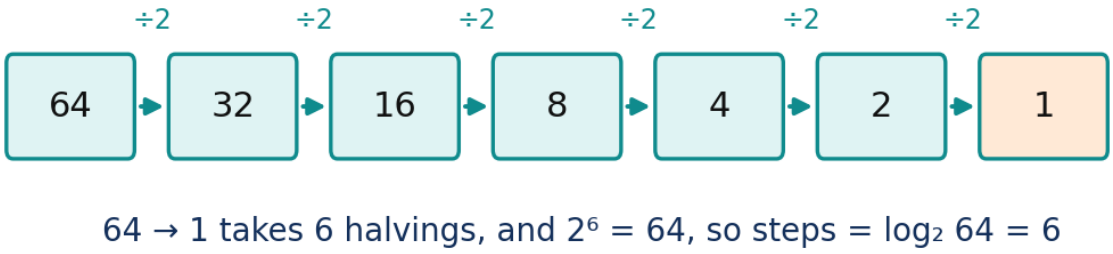


Figure 3. Halving 64 reaches 1 after $\log_2 64 = 6$ steps.

Operation counts (measured)

Table 3.3: iterations of the two loops

n	halving(n)	doubling(n)	$\lfloor \log_2 n \rfloor$	$\lceil \log_2 n \rceil$
1	0	0	0	0
2	1	1	1	1
4	2	2	2	2
8	3	3	3	3
16	4	4	4	4
64	6	6	6	6
256	8	8	8	8
1,024	10	10	10	10

The measured counts agree with the formulas $\lfloor \log_2 n \rfloor$ and $\lceil \log_2 n \rceil$ for all 11 values of n that we tested (1 to 1024).

Trace tables

Table 3.4: halving(40) — integer division, loop stops when n = 1

Iteration	n before	n after (n / 2)	count
1	40	20	1
2	20	10	2
3	10	5	3
4	5	2	4
5	2	1	5

Table 3.5: doubling(20) — i takes the values 1, 2, 4, 8, 16, 32

Iteration	i at the test	Test i < 20	count after
1	1	true	1
2	2	true	2
3	4	true	3
4	8	true	4
5	16	true	5
—	32	false (stop)	5

Time analysis

> **Derivation 3.1. Why log n?** After k halvings the size is $n / 2^k$. We stop when the size reaches about 1: $n / 2^k = 1$. Multiply both sides by 2^k: $n = 2^k$. Take log base 2 of both sides: $k = \log_2 n$. So the loop runs about $\log_2 n$ times. For the doubling loop, $i = 2^k$ after k doublings and the loop continues while $2^k < n$, i.e. while $k < \log_2 n$, so it also runs about $\log_2 n$ times.

Each iteration does a constant amount of work, so **T(n) = O(log n)**. To feel how small log n is: $\log_2 1{,}000 \approx 10$, $\log_2 1{,}000{,}000 \approx 20$, $\log_2 1{,}000{,}000{,}000 \approx 30$.

Auxiliary space

Two integers: **O(1)**. (A *recursive* halving function would use O(log n) stack space; see chapter 5.)

Where you meet it: binary search, balanced search trees, heap push/pop, fast exponentiation, counting the digits of a number.

3.3 O(n) — linear time

Analogy

Reading every name on an attendance list: one look per student. Twice as many students means twice as long.

Java code

Listing 3.3: linear loops

```
static int single(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        count++;                          // runs once per iteration
    }
    return count;                         // exactly n
}

static int sumArray(int[] a) {
    int sum = 0;                          // 1 step
    for (int x : a) {                     // n iterations
        sum += x;                         // 1 step each
    }
    return sum;                           // 1 step  -> total n + 2 = O(n)
}
```

Operation counts (measured)

Table 3.6: doubling n doubles the work

n	single(n) loop-body runs	sumArray: total steps (n + 2)
1	1	3
2	2	4
8	8	10
64	64	66
1,024	1,024	1,026

Trace table: sumArray on {7, 3, 9, 1, 5}

Table 3.7: the final sum 25 matches the measured result of running the method

i	x = a[i]	sum before	sum after
0	7	0	7
1	3	7	10
2	9	10	19
3	1	19	20
4	5	20	25

Time analysis

Initialising `sum` costs 1 step. The loop body (`sum += x`) costs 1 step and runs n times. The `return` costs 1 step. T(n) = n + 2. Drop the constant 2: **O(n)**. Variants such as `i += 2` (n/2 iterations), 2 passes (2n) and n + 100 all simplify to O(n).

Auxiliary space

`sum`, `i`, `x`: **O(1)**. The input array is not counted as auxiliary space.

3.4 O(n log n) — linearithmic time

Analogy

Sorting a deck of cards by splitting the pile in half again and again until single cards remain, then merging the small sorted piles back together. There are about log n levels of splitting, and at every level you handle all n cards once.

Java code

Listing 3.4: a loop of n iterations containing a loop of log n iterations

```
static int nLogN(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {         // outer: n times
        for (int j = 1; j < n; j *= 2) {  // inner: doubles, so about log2 n times
            count++;
        }
    }
    return count;                         // n * ceil(log2 n)
}
```

Operation counts (measured)

Table 3.8: the extra factor is only log n

n	nLogN(n) measured	n × ⌈log₂ n⌉	ratio to n
2	2	2	1
4	8	8	2
8	24	24	3
16	64	64	4
64	384	384	6
256	2,048	2,048	8
1,024	10,240	10,240	10

Trace table: nLogN(5)

Table 3.9: n = 5, inner loop j = 1, 2, 4 (three values)

i	j values tried (j < 5)	inner runs	running count
0	1, 2, 4	3	3
1	1, 2, 4	3	6
2	1, 2, 4	3	9
3	1, 2, 4	3	12
4	1, 2, 4	3	15

Total 15 = 5 × 3 = n × ⌈log₂ 5⌉.

Time analysis

The inner loop doubles `j` from 1 up to n, so it runs ⌈log₂ n⌉ times (Derivation 3.1). The outer loop runs n times. By the multiplication rule: n × ⌈log₂ n⌉, so **T(n) = O(n log n)**. Merge sort reaches the same bound differently: T(n) = 2T(n/2) + n (solved in chapter 6).

Auxiliary space

This loop uses **O(1)**. A top-down merge sort needs **O(n)** for its temporary arrays; heap sort needs O(1).

3.5 O(n²) — quadratic time

Analogy

A party of n guests where everyone shakes hands with everyone else. Each of n guests shakes about n hands, so roughly n² handshakes (exactly n(n−1)/2 distinct pairs). Doubling the guests quadruples the handshakes.

Java code

Listing 3.5: two nested loops over the same range

```
static int quadratic(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            count++;                        // n * n times
        }
    }
    return count;
}
```

Operation counts (measured)

Table 3.10: the triangular variant (j < i) is about half as much, but is still O(n²)

n	quadratic(n)	n²	triangular(n)	n(n−1)/2
1	1	1	0	0
2	4	4	1	1
4	16	16	6	6
8	64	64	28	28
16	256	256	120	120
32	1,024	1,024	496	496
64	4,096	4,096	2,016	2,016
128	16,384	16,384	8,128	8,128

Trace table: quadratic(3)

Table 3.11: 3 × 3 = 9 executions of count++

i	j	count after
0	0	1
0	1	2
0	2	3
1	0	4
1	1	5
1	2	6
2	0	7
2	1	8
2	2	9

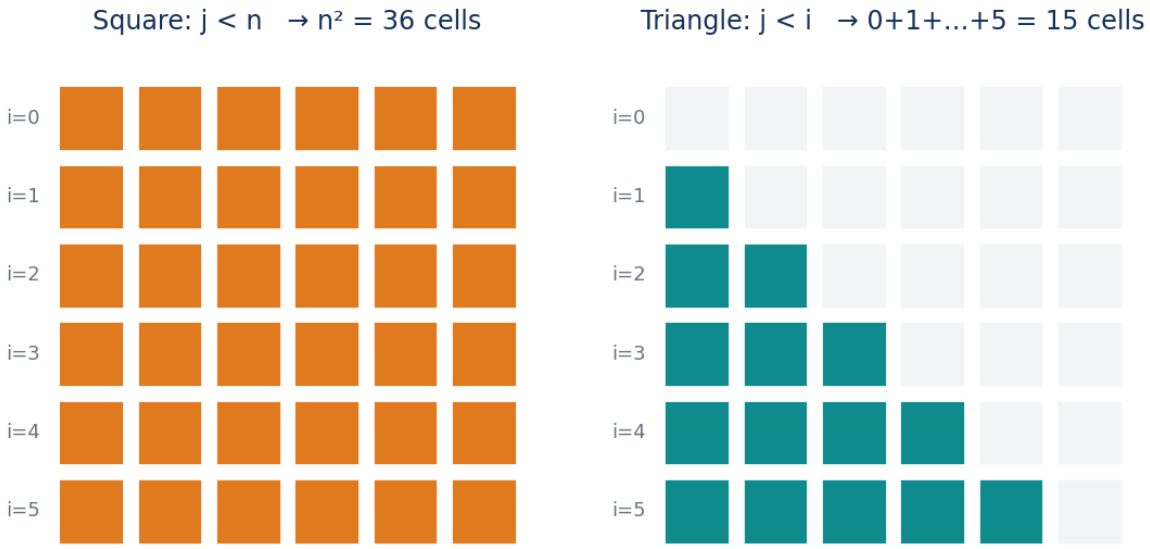


Figure 4. A square grid has n² cells; the triangular loop covers about half of them.

Time analysis

For each of n values of `i`, the inner loop runs n times: n · n = n². **T(n) = O(n²)**. Bubble sort, selection sort, insertion sort (worst case) and the "compare every pair" duplicate check are all quadratic.

Auxiliary space

O(1): `i`, `j`, `count`.

3.6 O(n³) — cubic time

Analogy

Checking every possible trio from n people for a three-way conflict: n choices for the first person, n for the second, n for the third. Doubling n makes the work 8 times larger.

Java code

Listing 3.6: three nested loops, and schoolbook matrix multiplication

```
static int cubic(int n) {
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            for (int k = 0; k < n; k++)
                count++;                  // n * n * n times
    return count;
}
static int[][] multiply(int[][] a, int[][] b) {        // both n x n
    int n = a.length;
    int[][] c = new int[n][n];                          // result: n*n extra cells
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            for (int k = 0; k < n; k++) {
                c[i][j] += a[i][k] * b[k][j];           // executed n * n * n times
            }
        }
    }
    return c;
}
```

Operation counts (measured)

Table 3.12: from n = 8 to n = 16 the count grows 8-fold

n	cubic(n)	n^3
1	1	1
2	8	8
4	64	64
8	512	512
16	4,096	4,096
32	32,768	32,768
64	262,144	262,144
100	1,000,000	1,000,000

Trace table: cubic(2)

Table 3.13: 2 × 2 × 2 = 8

i	j	k	count after
0	0	0	1
0	0	1	2
0	1	0	3
0	1	1	4
1	0	0	5
1	0	1	6
1	1	0	7
1	1	1	8

Time analysis

$n \cdot n \cdot n = n^3$, so **T(n) = O(n³)**. Multiplying two n × n matrices with three nested loops performs exactly n^3 multiplications (8 for 2 × 2, 27 for 3 × 3).

Auxiliary space

`cubic` uses O(1). `multiply` allocates the result matrix `c`, which is **O(n²)** (output space; see chapter 5).

3.7 O(2ⁿ) — exponential time

Analogy

Packing a suitcase from n items: each item is either in or out. Two choices per item, so 2^n possible suitcases. One extra item doubles the number of suitcases to consider.

Java code

Listing 3.7: naive recursive Fibonacci and subset enumeration

```
static int fibCalls = 0;

static int fib(int n) {
    fibCalls++;                          // count every call
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);      // two recursive calls per call
}
static long subsetWork(int n) {
    long count = 0;
    for (int mask = 0; mask < (1 << n); mask++) {   // 2^n subsets
        for (int bit = 0; bit < n; bit++) {         // inspect n bits of each mask
            if ((mask & (1 << bit)) != 0) {
                count++;                            // element 'bit' is in this subset
            }
        }
    }
    return count;                        // total elements written = n * 2^(n-1)
}
```

Operation counts (measured)

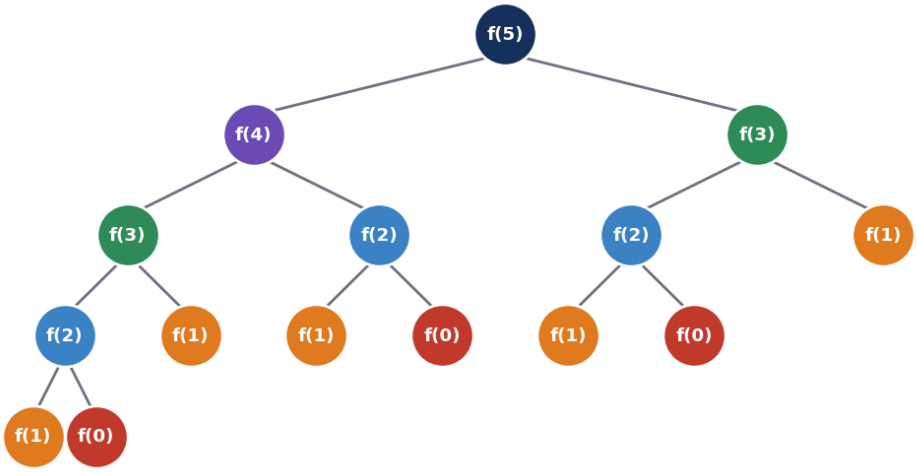
Table 3.14: naive Fibonacci makes 2,692,537 calls to compute fib(30)

n	fib(n) value	calls made (measured)	2·F(n+1) − 1	2ⁿ
1	1	1	1	2
2	1	3	3	4
3	2	5	5	8
4	3	9	9	16
5	5	15	15	32
10	55	177	177	1,024
15	610	1,973	1,973	32,768
20	6,765	21,891	21,891	1,048,576
25	75,025	242,785	242,785	33,554,432
30	832,040	2,692,537	2,692,537	1,073,741,824

Trace table: the 9 calls of fib(4), in order

Table 3.15: fib(2) is computed twice, fib(1) three times, fib(0) twice

Call #	Call (indent = depth)	Returns
1	fib(4)	3
2	fib(3)	2
3	fib(2)	1
4	fib(1)	1
5	fib(0)	0
6	fib(1)	1
7	fib(2)	1
8	fib(1)	1
9	fib(0)	0



fib(5) makes 15 calls. f(3) is computed 2 times, f(2) 3 times, f(1) 5 times. Same colour = same repeated work.

Figure 5. The call tree of fib(5). Repeated colours show repeated work; that repetition is what memoization removes.

Time analysis

Derivation 3.2. How many calls? Let C(n) be the number of calls made by fib(n). The call itself counts 1, then it makes calls for n−1 and n−2, so C(n) = 1 + C(n−1) + C(n−2), with C(0) = C(1) = 1. Check: C(2) = 1+1+1 = 3, C(3) = 1+3+1 = 5, C(4) = 1+5+3 = 9 ✓ (matches the table). **Upper bound:** since C(n−2) ≤ C(n−1), we get C(n) ≤ 2C(n−1) + 1. Unroll: C(n) ≤ 2(2C(n−2)+1)+1 = 4C(n−2) + 3 ≤ 8C(n−3) + 7 ≤ … ≤ $2^k \cdot C(n-k) + (2^k - 1)$. With k = n: C(n) ≤ $2^n \cdot C(0) + 2^n - 1 = 2^{n+1} - 1$. So fib is O($2^n$). (A tighter bound is about 1.618^n, since C(n) = 2F(n+1) − 1.)

The subset loop runs 2^n masks and inspects n bits each: **O(n · 2^n)**. Measured: the total number of set bits over all masks is n · 2^{n-1} (for n = 10 the measured value is 5,120, and 10 · 2^9 = 5,120 ✓).

Auxiliary space

Key idea. Exponential **time** does not mean exponential **space**. The recursion of `fib(n)` never has more than n calls alive at once (a single root-to-leaf path), so its stack space is **O(n)**. The subset loop uses O(1) extra space apart from any output.

3.8 O(n!) — factorial time

Analogy

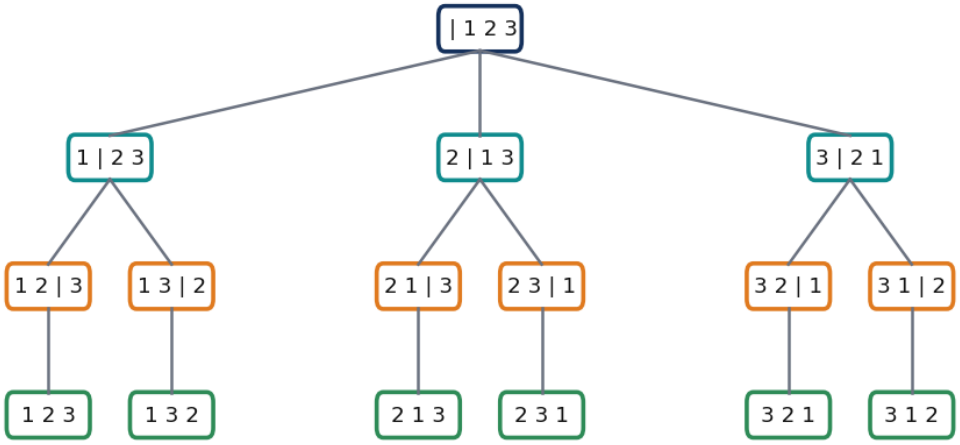
Seating n people in a row in every possible order: n choices for the first seat, n − 1 for the second, then n − 2, down to 1. The number of orders is n × (n−1) × … × 1 = n!.

Java code

Listing 3.8: generating all permutations by swapping and backtracking

```
static long leaves = 0, calls = 0;

static void permute(int[] a, int k) {
    calls++;
    if (k == a.length) {                  // a complete permutation is ready
        leaves++;
        return;
    }
    for (int i = k; i < a.length; i++) {
        swap(a, k, i);                    // choose a[i] for position k
        permute(a, k + 1);                // arrange the rest
        swap(a, k, i);                    // undo the choice (backtrack)
    }
}
```



Levels: 1 + 3 + 6 + 6 = 16 calls; the last level holds 3! = 6 complete permutations

Figure 6. The call tree of permute for {1, 2, 3}: 16 calls, 6 complete permutations at the bottom.

Operation counts (measured)

Table 3.16: n = 9 already needs 986,410 calls

n	permutations (leaves)	n!	total calls (measured)	Σ n!/k!
1	1	1	2	2
2	2	2	5	5
3	6	6	16	16
4	24	24	65	65
5	120	120	326	326
6	720	720	1,957	1,957
7	5,040	5,040	13,700	13,700
8	40,320	40,320	109,601	109,601
9	362,880	362,880	986,410	986,410

Trace table: the leaves of permute({1,2,3}) in order

Table 3.17

Leaf #	Permutation reached
1	{1, 2, 3}
2	{1, 3, 2}
3	{2, 1, 3}
4	{2, 3, 1}
5	{3, 2, 1}
6	{3, 1, 2}

Time analysis

Derivation 3.3. Counting leaves. Position 0 can be filled in n ways; for each, position 1 in n−1 ways; … ; position n−1 in 1 way. By the multiplication rule: n · (n−1) · … · 1 = n! leaves. **Counting all calls.** At depth k there are n!/(n−k)! calls (the number of ways to fill the first k positions). Total = Σ for k = 0..n of n!/(n−k)!. Substitute j = n − k: n! · Σ for j = 0..n of 1/j!. The series Σ 1/j! approaches e ≈ 2.718, so the total is about e · n!. Check n = 3: 1 + 3 + 6 + 6 = 16 ✓ and e · 6 ≈ 16.3 ✓. So the number of calls is Θ(n!). If you also copy or print each permutation (n items), the full cost is O(n · n!).

Auxiliary space

Recursion depth is n + 1 and the array is permuted in place, so the extra space is **O(n)** (excluding any stored output; storing all permutations would need n · n! cells).

Interview tip. Practical limit: n! reaches 40 million at n = 11 and 479 million at n = 12. Factorial algorithms are only feasible for n ≤ 10 or 11 unless you prune the search (backtracking with pruning).

3.9 Chapter summary

Table 3.18: summary

Class	Time example	Doubling n makes the work…	Aux. space of the example
O(1)	swap two elements	unchanged	O(1)
O(log n)	halving loop, binary search	increase by 1 step	O(1) iterative
O(n)	sum an array	2× larger	O(1)
O(n log n)	merge sort	a bit more than 2×	O(n) for merge sort
$O(n^2)$	compare all pairs	4×	O(1)
$O(n^3)$	three nested loops	8×	O(1) (+ $O(n^2)$ output)
$O(2^n)$	naive Fibonacci, all subsets	squared ($2^{2n} = (2^n)^2$)	O(n) stack
O(n!)	all permutations	multiplied by ≈ (2n)!/n! — huge	O(n) stack

4. The Loop Analysis Toolbox

Almost every complexity question reduces to counting how many times the innermost statement runs. This chapter covers nine loop shapes. In each case the counts shown as "measured" were produced by running the Java code and are compared with the derived formula.

> **Key idea. Four-step recipe.** (1) Find the innermost statement. (2) For each loop, write how many times it runs, in terms of the enclosing loop variables. (3) Multiply for nested loops (summing when the bound depends on an outer variable), add for loops in sequence. (4) Simplify with the rules of chapter 2.

4.1 Single loops

Table 4.1: stepping by a constant never changes the class

Loop header	Iterations	Big-O
`for (i = 0; i < n; i++)`	n	O(n)
`for (i = 0; i < n; i += 2)`	⌈n / 2⌉	O(n)
`for (i = 0; i < n; i += 3)`	⌈n / 3⌉	O(n)
`for (i = n; i > 0; i--)`	n	O(n)
`for (i = 0; i < 1000; i++)`	1000 (does not depend on n)	O(1)

> **Derivation 4.1. Why ⌈n/3⌉?** With `i += 3` the values are 0, 3, 6, …, 3(k−1). The loop continues while 3(k−1) < n, so k − 1 < n/3, so k < n/3 + 1. The number of iterations is the smallest integer k satisfying the last inequality, namely k = ⌈n/3⌉. For n = 10: 0, 3, 6, 9 → 4 = ⌈10/3⌉ ✓ (Exercise 1 checks this by running the code).

4.2 Nested loops

When the inner bounds do not depend on the outer variable, multiply. Different bounds stay different:

```
for (int i = 0; i < n; i++)          // n times
    for (int j = 0; j < m; j++)      // m times for each i
        work();                       // n * m times in total  -> O(n * m)
```

With n = m this is O(n²). The cost of a nested loop is the *product* of the iteration counts, never the sum (a sum arises only when the loops come one after the other).

4.3 Separate (sequential) loops

Listing 4.1: loops one after the other

```
static int separate(int n) {              // two loops, one after the other
    int count = 0;
    for (int i = 0; i < n; i++) count++;  // n
    for (int j = 0; j < n; j++) count++;  // n
    return count;                         // 2n  -> O(n)
}

static int separateMixed(int n) {         // a linear loop, then a nested loop
    int count = 0;
    for (int i = 0; i < n; i++) count++;              // n
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) count++;          // n^2
    return count;                         // n + n^2  -> O(n^2)
}
```

Table 4.2: measured counts

n	separate(n) = 2n	separateMixed(n) = n + n²
8	16	72
64	128	4,160
1,024	2,048	1,049,600

Sequential code adds. 2n is O(n). n + n² is O(n²): the larger term wins, and for n = 1024 the linear part (1,024) is a tiny fraction of the total (1,049,600).

4.4 Constant-bound inner loops

Listing 4.2

```
static int constInner(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {         // n times
        for (int j = 0; j < 5; j++) {     // always exactly 5 times
            count++;
        }
    }
    return count;                         // 5n  -> O(n)
}
```

Table 4.3

n	constInner(n) measured	5n
8	40	40
64	320	320
1,024	5,120	5,120

The inner loop is a constant (5 iterations), so the total is 5 · n: **O(n)**. Even if the inner loop ran 1,000 times, the class would still be O(n); but a constant of 1,000 matters in practice, which is why Big-O is a tool for comparing growth, not for predicting exact running time.

4.5 Doubling and halving loops

Table 4.4

Loop	Variable takes the values	Iterations
`for (i = 1; i < n; i *= 2)`	1, 2, 4, 8, …	⌈log₂ n⌉
`for (i = 1; i < n; i *= 3)`	1, 3, 9, 27, …	⌈log₃ n⌉
`for (i = n; i > 1; i /= 2)`	n, n/2, n/4, …	⌊log₂ n⌋
`for (i = 2; i < n; i = i * i)`	2, 4, 16, 256, 65536, …	about $\log_2 \log_2 n$ ($n = 2^{16} = 65{,}536$ gives 4 iterations)

All the first three are O(log n), since $\log_3 n = \log_2 n / \log_2 3$ (a constant multiple). The measured counts for `halving` and `doubling` appear in Table 3.3.

4.6 Triangular loops

Listing 4.3: the inner bound depends on i

```
static int triangular(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {     // inner bound depends on i
            count++;
        }
    }
    return count;                         // 0 + 1 + 2 + ... + (n-1) = n(n-1)/2
}
```

Derivation 4.2. Gauss's trick. The body runs 0 + 1 + 2 + ... + (n−1) times. Call this sum S. Write it forwards and backwards:
S = 0 + 1 + 2 + ... + (n−2) + (n−1)
S = (n−1) + (n−2) + ... + 1 + 0
Add the two lines column by column. Each column sums to n − 1, and there are n columns, so 2S = n(n−1). Therefore **S = n(n−1)/2**. For n = 10: 10·9/2 = 45. The same trick shows 1 + 2 + ... + n = n(n+1)/2. Since n(n−1)/2 = n²/2 − n/2, the dominant term is n²/2, and dropping the constant ½ gives **O(n²)**.

Table 4.5: measured vs formula

n	triangular(n) measured	n(n−1)/2
1	0	0
2	1	1
3	3	3
4	6	6
5	10	10
8	28	28
10	45	45
16	120	120
100	4,950	4,950

4.7 Three nested loops

Independent bounds give n · n · n = n³ (Listing 3.6). With the bounds tied together (each inner loop runs up to the variable of the loop outside it), the count shrinks by a constant factor but the class stays the same.

Listing 4.4

```
static int tripleTriangular(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= i; j++)
            for (int k = 1; k <= j; k++)
                count++;                  // n(n+1)(n+2)/6
    return count;
}
```

Derivation 4.3. Counting n(n+1)(n+2)/6. For fixed i, the two inner loops execute Σ for j = 1..i of j = i(i+1)/2 times (Gauss). So the total is T = Σ for i = 1..n of i(i+1)/2 = ½(Σ i² + Σ i). Use Σ i = n(n+1)/2 and Σ i² = n(n+1)(2n+1)/6 (derived in section 4.8). Then T = ½ · [n(n+1)(2n+1)/6 + n(n+1)/2]. Let A = n(n+1) to shorten the algebra. Put both fractions over 6: T = ½ · [A(2n+1)/6 + 3A/6] = ½ · A(2n+1+3)/6 = ½ · A(2n+4)/6 = A(2n+4)/12 = A(n+2)/6. So **T = n(n+1)(n+2)/6**. For n = 3 this is 3·4·5/6 = 10 ✓. This is n³/6 + lower terms, so the class is **O(n³)**.

Table 4.6: the ratio tends to 1/6 ≈ 0.167

n	tripleTriangular(n) measured	n(n+1)(n+2)/6	n³	measured ÷ n³
1	1	1	1	1.000
2	4	4	8	0.500
3	10	10	27	0.370
4	20	20	64	0.312
5	35	35	125	0.280
8	120	120	512	0.234
10	220	220	1,000	0.220
16	816	816	4,096	0.199
64	45,760	45,760	262,144	0.175
100	171,700	171,700	1,000,000	0.172

4.8 Dependent loops

A loop is **dependent** when its bound uses a variable of an enclosing loop. Then you must *sum* over the outer variable instead of multiplying.

Case A: the inner bound is i²

Listing 4.5

```
static int dependentSquare(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i * i; j++) {   // inner bound is i squared
            count++;
        }
    }
    return count;                          // 1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
}
```

Derivation 4.4. Deriving Σ k² = n(n+1)(2n+1)/6 with no steps skipped. Start from the identity (k+1)³ − k³ = 3k² + 3k + 1 (expand (k+1)³ = k³ + 3k² + 3k + 1). Sum it for k = 1 to n. The left side telescopes (each middle term cancels): (n+1)³ − 1³ = n³ + 3n² + 3n. The right side is 3Σk² + 3Σk + n. So 3Σk² = n³ + 3n² + 3n − 3·n(n+1)/2 − n = n³ + 3n² + 2n − (3n² + 3n)/2. Put everything over 2: = (2n³ + 6n² + 4n − 3n² − 3n)/2 = (2n³ + 3n² + n)/2 = n(2n² + 3n + 1)/2 = n(n+1)(2n+1)/2. Divide by 3: **Σ k² = n(n+1)(2n+1)/6**. Check n = 3: 1 + 4 + 9 = 14 and 3·4·7/6 = 14 ✓.

The body of `dependentSquare` runs Σ i² times, which is about n³/3. **O(n³)**. Measured: n = 10 gives 385 (formula 10·11·21/6 = 385 ✓) and n = 100 gives 338,350.

Case B: the outer variable halves or doubles

Listing 4.6: a surprise — these are O(n), not O(n log n)

```
static int halvingOuter(int n) {
    int count = 0;
    for (int i = n; i > 0; i = i / 2) {   // i = n, n/2, n/4, ...
        for (int j = 0; j < i; j++) {     // inner runs i times
            count++;
        }
    }
    return count;                         // n + n/2 + n/4 + ... < 2n  -> O(n)
}

static int doublingOuter(int n) {
    int count = 0;
    for (int i = 1; i < n; i = i * 2) {   // i = 1, 2, 4, 8, ...
        for (int j = 0; j < i; j++) {
            count++;
        }
    }
    return count;                         // 1 + 2 + 4 + ... < 2n  -> O(n)
}
```

Table 4.7: measured

n	halvingOuter(n)	2n − 1	doublingOuter(n)	2^⌈log₂ n⌉ − 1
8	15	15	7	7
64	127	127	63	63
1,024	2,047	2,047	1,023	1,023

Derivation 4.5. Geometric series. `halvingOuter` runs n + n/2 + n/4 + … + 1 times (for n a power of two). Factor out n: n(1 + ½ + ¼ + … + 1/n). A geometric series with ratio r = ½ sums to $(1 - r^m)/(1 - r)$ which is less than $1/(1 - ½) = 2$. So the total is less than 2n: **O(n)**. For n = 16: 16 + 8 + 4 + 2 + 1 = 31 = 2n − 1 ✓ (matches the measured pattern). The doubling version counts $1 + 2 + 4 + \ldots + 2^k = 2^{k+1} - 1$ with $2^k < n$, again less than 2n: **O(n)**.

Watch out. "An outer loop of log n iterations containing an inner loop of up to n iterations" is **not** automatically n log n. Always sum the actual inner counts; here the inner counts form a geometric series that adds up to O(n).

Case C: a harder dependent loop (interview favourite)

Listing 4.7

```
static long dependentHard(int n) {
    long count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i * i; j++) {
            if (j % i == 0) {             // true only for j = i, 2i, ..., i*i
                for (int k = 0; k < j; k++) {
                    count++;
                }
            }
        }
    }
    return count;                         // grows like n^4 / 8
}
```

Derivation 4.6. Counting the innermost increments. Fix i. The values of j divisible by i inside 1..i² are j = i·1, i·2, …, i·i (that is i values). For j = i·m the k-loop runs j = i·m times. So for this i the count is Σ for m = 1..i of i·m = i · i(i+1)/2 = i²(i+1)/2. Sum over i: T = ½ Σ i²(i+1) = ½ (Σ i³ + Σ i²). With Σ i³ = [n(n+1)/2]² and Σ i² = n(n+1)(2n+1)/6: T = ½ (n²(n+1)²/4 + n(n+1)(2n+1)/6). The leading term is n⁴/8, so **T = O(n⁴)**. Check n = 10: ½(3025 + 385) = 1,705 ✓ and n = 100: ½(25,502,500 + 338,350) = 12,920,425 ✓ (both measured by the Java code: 1,705 and 12,920,425). Note that the middle loop itself costs Σ i² = O(n³) test iterations, which is smaller than n⁴, so it does not change the class.

4.9 Loops with early breaks

A `break` or `return` inside a loop shortens it for some inputs. Big-O quotes the **worst** case, so the question is: can an adversary force the loop to run to the end?

Listing 4.8

```
static int linearSearchSteps(int[] a, int target) {
    int steps = 0;
    for (int i = 0; i < a.length; i++) {
        steps++;                          // one comparison
        if (a[i] == target) break;        // early exit
    }
    return steps;                         // best 1, worst n
}
```

Table 4.8

Search in {7, 3, 9, 1, 5}	Steps (measured)	Case
target 7 (first)	1	best case, O(1)
target 9 (middle)	3	average-like
target 5 (last)	5	worst case, O(n)
target 42 (absent)	5	worst case, O(n)

Now an early break inside a nested loop:

Listing 4.9

```
static int nestedBreak(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i * j >= n) break;        // leave the inner loop as soon as i*j reaches n
            count++;
        }
    }
    return count;                         // about n * (1 + 1/2 + 1/3 + ...) = O(n log n)
}
```

Derivation 4.7. Counting. For i = 0 the test i·j ≥ n is never true (it is 0), so the inner loop runs n times. For i ≥ 1 the inner loop continues while i·j < n, i.e. j < n/i, so the body runs ⌈n/i⌉ times. Total = n + Σ for i = 1..n−1 of ⌈n/i⌉. Since n/i ≤ ⌈n/i⌉ < n/i + 1, the sum lies between n·H and n·H + n, where H = 1 + ½ + ⅓ + … + 1/(n−1) is the harmonic number. H grows like ln n (proved by comparing the sum with the integral of 1/x). Hence the total is about n ln n = n · (log₂ n / log₂ e), which is **O(n log n)**. For n = 100: measured 672 versus n ln n = 461 (the exact value n + Σ⌈n/i⌉ is bigger by an amount of order n, as expected).

4.10 Pattern summary

Table 4.9: loop shapes, formulas, and counts measured with n = 64

Pattern	Code shape	Count formula	Measured at n = 64	Big-O
single loop	`for i<n`	n	64	O(n)
separate loops	`for i<n` twice	2n	128	O(n)
constant inner	`j < 5` inside `i < n`	5n	320	O(n)
doubling inner	`j *= 2` inside `i < n`	n⌈log₂ n⌉	384	O(n log n)
halving outer, i inner	`i = n; i/=2` and `j < i`	2n − 1	127	O(n)
nested	`i < n`, `j < n`	n²	4,096	O(n²)
triangular	`j < i`	n(n−1)/2	2,016	O(n²)
three nested	`i, j, k < n`	n³	262,144	O(n³)
triple triangular	`j ≤ i`, `k ≤ j`	n(n+1)(n+2)/6	45,760	O(n³)
dependent i²	`j ≤ i*i`	n(n+1)(2n+1)/6	89,440	O(n³)
hard dependent	Listing 4.7	½(Σi³ + Σi²)	2,207,920	O(n⁴)
early break nested	Listing 4.9	n + Σ⌈n/i⌉	400	O(n log n)

5. Space Complexity

5.1 Total space versus auxiliary space

Total space is all the memory an algorithm uses, including the input. **Auxiliary space** is only the *extra* memory it allocates while running: variables, temporary arrays, hash tables, and the recursion stack. When people say "space complexity" in an interview they almost always mean auxiliary space, but you should say which you mean.

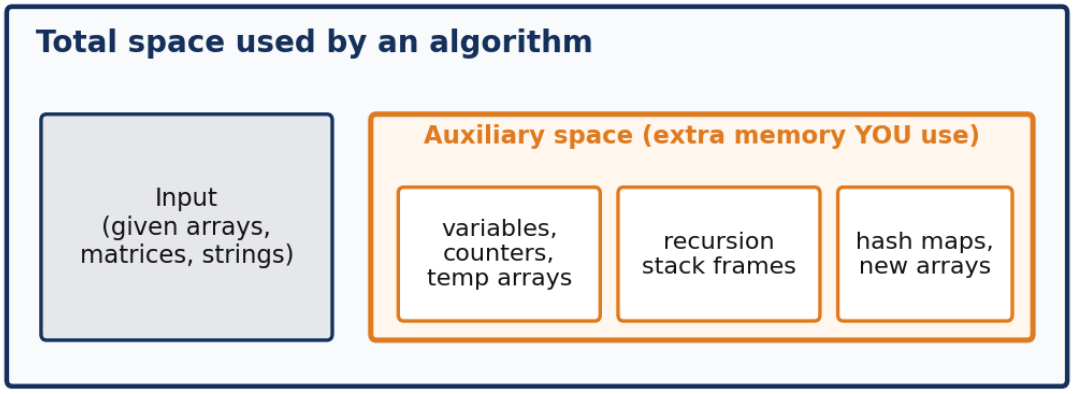


Figure 7. Auxiliary space = variables + recursion stack + any new data structures you create.

Table 5.1

Counts as auxiliary space	Does not count
Extra arrays, lists, maps, sets you create	The input array or string you were given
Recursion stack frames (depth × frame size)	The array you modify in place (no new storage)
Temporary copies, such as `substring` or `clone()`	The fixed code of the program
The output, *if* you state that you count it	—

> **Interview tip.** Say it out loud: "Auxiliary space is O(1) not counting the input; if we count the output array it is O(n)." Interviewers like it when you state your convention.

5.2 Variables and fixed-size structures: O(1)

Primitive variables (`int`, `long`, `double`, `boolean`, `char`) take a fixed number of bytes. Ten counters or a `new int[26]` frequency table are still O(1), because their size does not depend on `n`. (That is why the first-unique-character solution in chapter 9 is O(1) space.)

5.3 Arrays and matrices

- `new int[n]` is O(n) space (about 4n bytes plus a small header).
- `new int[n][m]` is O(n · m). A square n × n matrix is O(n^2).
- A one-dimensional prefix-sum array is O(n); overwriting the input instead is O(1).

Listing 5.1: prefix sums with new storage versus in place

```
static int[] prefixSums(int[] a) {     // input O(n) is not counted; output array is O(n)
    int[] p = new int[a.length];
    int run = 0;
    for (int i = 0; i < a.length; i++) { run += a[i]; p[i] = run; }
    return p;
}

static void prefixSumsInPlace(int[] a) {       // overwrites the input: auxiliary space O(1)
    for (int i = 1; i < a.length; i++) a[i] += a[i - 1];
}
```

Listing 5.2: transposing a matrix in place versus into a new matrix

```
static void transposeInPlace(int[][] m) {      // square n x n matrix, auxiliary space O(1)
    int n = m.length;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            int t = m[i][j]; m[i][j] = m[j][i]; m[j][i] = t;
        }
    }
}

static int[][] transposeCopy(int[][] m) {      // auxiliary space O(rows * cols)
    int rows = m.length, cols = m[0].length;
    int[][] t = new int[cols][rows];
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            t[j][i] = m[i][j];
    return t;
}
```

In-place transpose swaps each pair `m[i][j]` and `m[j][i]` once (only for j > i), using O(1) extra space and about n²/2 swaps. The copy version needs a whole second matrix: O(rows × cols).

5.4 In-place algorithms

An algorithm is **in place** if it transforms the input using O(1) extra space (some authors allow O(log n), for example for a quick sort stack).

Listing 5.3: reversing an array in place versus with a copy

```
static void reverseInPlace(int[] a) {  // auxiliary space O(1)
    for (int i = 0, j = a.length - 1; i < j; i++, j--) {
        int t = a[i]; a[i] = a[j]; a[j] = t;
    }
}

static int[] reverseCopy(int[] a) {    // auxiliary space O(n): a second array of size n
    int[] out = new int[a.length];
    for (int i = 0; i < a.length; i++) out[a.length - 1 - i] = a[i];
    return out;
}
```

Table 5.2

Sorting algorithm	Auxiliary space	In place?
Bubble, selection, insertion	O(1)	yes
Heap sort	O(1)	yes
Quick sort	O(log n) stack on average, O(n) worst	yes (partitioning is in place)
Merge sort (array version)	O(n) temporary array	no

5.5 The recursion stack

Every method call creates a **stack frame** that stores its parameters, its local variables and where to return to. The frames of all calls that are *still waiting* sit on the stack at the same time. So the space used by recursion is (maximum depth) × (frame size), which in Big-O is just the maximum depth.

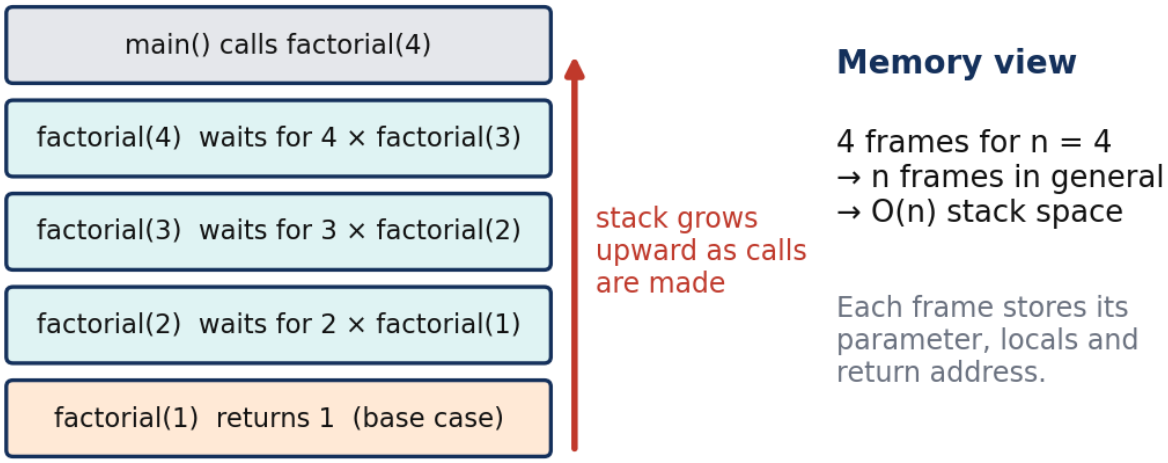


Figure 8. factorial(4) has 4 frames alive at the deepest point. That is O(n) stack space for factorial(n).

Listing 5.4: depth n versus depth log n

```
static int factorial(int n) {          // depth n, so stack space O(n)
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

static int power(int x, int n) {       // fast power: depth log2 n, so stack space O(log n)
    if (n == 0) return 1;
    int half = power(x, n / 2);
    return (n % 2 == 0) ? half * half : half * half * x;
}
```

Table 5.3: stack depth, checked by running the code

Call	Depth of the stack	Value
`factorial(5)`	5 frames: n = 5, 4, 3, 2, 1	120
`power(3, 5)`	4 frames: n = 5, 2, 1, 0	243
`power(2, 10)`	5 frames: n = 10, 5, 2, 1, 0	1024

Derivation 5.1. Why does power() have depth about log n? Each call passes n/2 (integer division) to the next call, stopping at 0. Starting from n, the arguments are n, ⌊n/2⌋, ⌊n/4⌋, …, 1, 0. Reaching 1 takes ⌊log₂ n⌋ halvings, and one more call reaches 0, giving ⌊log₂ n⌋ + 2 frames. For n = 5: ⌊2.32⌋ + 2 = 4 ✓. For n = 10: 3 + 2 = 5 ✓. So the stack space is O(log n).

Listing 5.5: the same sum, two space costs

```
static int sumIter(int[] a) {          // time O(n), auxiliary space O(1)
    int sum = 0;
    for (int x : a) sum += x;
    return sum;
}

static int sumRec(int[] a, int i) {    // time O(n), auxiliary space O(n): n stack frames
    if (i == a.length) return 0;
    return a[i] + sumRec(a, i + 1);
}
```

Both versions run in O(n) time. `sumIter` uses O(1) space; `sumRec` uses O(n) stack space and would throw `StackOverflowError` for a large enough array (the default Java stack handles roughly ten thousand frames of a simple method, depending on the JVM settings).

> **Watch out.** Binary search written *recursively* is O(log n) space; written with a loop it is O(1). Merge sort has O(log n) stack frames *and* an O(n) temporary array, so its auxiliary space is O(n) in total.

5.6 Java-specific space notes

Table 5.4

Operation	Extra space	Comment
`s.substring(i, j)`	O(j − i)	Since Java 7u6 it copies the characters.
`s.toCharArray()`	O(n)	New array.
`new StringBuilder()` of n appends	O(n)	Resizes by doubling.
`HashSet` / `HashMap` with n entries	O(n)	Each entry is an object; the constant is large.
`int[] copy = a.clone()`	O(n)	Full copy.
`Arrays.sort(int[])`	O(log n) typical	Dual-pivot quick sort on primitives.
`Collections.sort(list)`	O(n)	TimSort uses a merge buffer.

5.7 The time–space trade-off

Extra memory can often buy speed. The same problem appears with different costs:

Table 5.5

Problem	Low-space solution	Fast solution
Duplicate in an array	nested loops: O(n^2) time, O(1) space	`HashSet`: O(n) time, O(n) space
Fibonacci(n)	naive recursion: O(2^n) time, O(n) stack	memo (below): O(n) time, O(n) space; loop: O(n) time, O(1) space
Sum of a range, many queries	recompute: O(n) per query	prefix sums: O(n) build, O(1) per query, O(n) space
Two-sum	brute force: O(n^2), O(1)	`HashMap`: O(n), O(n)

Listing 5.6: memoised and iterative Fibonacci (both verified against each other)

```
static long[] memo = new long[91];                // fib(90) fits in a long

static long fibMemo(int n) {                       // time O(n), space O(n)
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];              // already computed: O(1)
    memo[n] = fibMemo(n - 1) + fibMemo(n - 2);     // each n is computed once
    return memo[n];
}

static long fibIter(int n) {                       // time O(n), space O(1)
    long prev = 0, cur = 1;
    for (int i = 0; i < n; i++) {
        long next = prev + cur;
        prev = cur;
        cur = next;
    }
    return prev;
}
```

6. Recursion and Recurrences

A recursive method calls itself on smaller inputs. Its running time satisfies a **recurrence**: an equation that writes T(n) in terms of T of smaller inputs. We solve it by **unrolling** (substituting the recurrence into itself until the base case) or by drawing the recursion tree.

6.1 T(n) = T(n − 1) + c → O(n)

Example: `factorial(n)` does constant work, then makes one call with n − 1.

Unrolling 6.1. T(n) = T(n−1) + c = [T(n−2) + c] + c = T(n−2) + 2c = T(n−3) + 3c = … = T(n−k) + k·c. Stop when n − k = 0, i.e. k = n: T(n) = T(0) + n·c. With T(0) a constant, **T(n) = O(n)**. Recursion tree: a single chain of n nodes, each doing c work.

6.2 T(n) = T(n / 2) + c → O(log n)

Example: recursive binary search, fast power.

Unrolling 6.2. T(n) = T(n/2) + c = T(n/4) + 2c = T(n/8) + 3c = … = $T(n/2^k)$ + k·c. The input becomes 1 when $n/2^k = 1$, i.e. $k = \log_2 n$. So T(n) = T(1) + c·$\log_2 n$ = **O(log n)**.

6.3 T(n) = 2T(n / 2) + n → O(n log n)

Example: merge sort. Two half-size sorts, plus a linear merge.

Unrolling 6.3. Write the merge cost as c·n. T(n) = 2T(n/2) + cn. Substitute for T(n/2) = 2T(n/4) + c·n/2: T(n) = 2[2T(n/4) + c·n/2] + cn = 4T(n/4) + cn + cn = 4T(n/4) + 2cn. Again, T(n/4) = 2T(n/8) + c·n/4: T(n) = 4[2T(n/8) + cn/4] + 2cn = 8T(n/8) + 3cn. The pattern after k steps: T(n) = 2^k·T($n/2^k$) + k·cn. Stop when $n/2^k = 1$, so $2^k = n$ and $k = \log_2 n$: T(n) = n·T(1) + cn·$\log_2 n$ = **O(n log n)**. Tree view: level j has 2^j subproblems of size $n/2^j$, so every level costs $2^j \cdot c \cdot n/2^j$ = cn, and there are $\log_2 n + 1$ levels.

6.4 T(n) = 2T(n − 1) + c → O(2^n)

Example: naive Fibonacci (upper bound), towers of Hanoi, generating all subsets by recursion.

Unrolling 6.4. T(n) = 2T(n−1) + c = 2[2T(n−2) + c] + c = 4T(n−2) + 3c = 8T(n−3) + 7c. After k steps: T(n) = 2^k·T(n−k) + (2^k − 1)·c. At k = n: T(n) = 2^n·T(0) + (2^n − 1)·c = **O(2^n)**.

6.5 T(n) = T(n − 1) + n → O(n^2)

Example: selection sort viewed recursively; quick sort when every partition is as unbalanced as possible.

Unrolling 6.5. T(n) = T(n−1) + n = T(n−2) + (n−1) + n = … = T(0) + 1 + 2 + … + n = T(0) + n(n+1)/2 (Gauss). So **T(n) = O(n^2)**.

6.6 T(n) = 2T(n / 2) + c → O(n)

Example: ex5 in the final exercises. Two half-size calls and constant work (like visiting every node of a balanced binary tree).

> **Unrolling 6.6.** Level j of the recursion tree has 2^j calls, each doing c. The tree has $\log_2 n + 1$ levels, so the total is c·(1 + 2 + 4 + … + 2^(log₂ n)) = c·(2^(log₂ n + 1) − 1) = c·(2n − 1). So **T(n) = O(n)**. (For n a power of two the code makes exactly 2n − 1 calls.)

6.7 The Master Theorem (shortcut)

For recurrences of the form $T(n) = a \cdot T(n/b) + \Theta(n^d)$ with $a \geq 1$, $b > 1$, $d \geq 0$, compare a with b^d:

Table 6.1

Case	Condition	Result	Intuition
1	$a < b^d$	$\Theta(n^d)$	the top-level work dominates
2	$a = b^d$	$\Theta(n^d \cdot \log n)$	every level costs the same
3	$a > b^d$	Θ(n^(log_b a))	the leaves dominate

Table 6.2: applying the theorem

Algorithm	Recurrence	a, b, d	Case	Result
Binary search	T(n) = T(n/2) + 1	1, 2, 0	$a = b^d$ (1 = 1)	Θ(log n)
Merge sort	T(n) = 2T(n/2) + n	2, 2, 1	$a = b^d$ (2 = 2)	Θ(n log n)
Tree traversal	T(n) = 2T(n/2) + 1	2, 2, 0	$a > b^d$ (2 > 1)	Θ(n)
Search then scan	T(n) = T(n/2) + n	1, 2, 1	$a < b^d$ (1 < 2)	Θ(n)
Karatsuba multiplication	T(n) = 3T(n/2) + n	3, 2, 1	$a > b^d$ (3 > 2)	Θ(n^1.585)

> **Watch out.** The Master Theorem does not apply to T(n) = T(n−1) + …, which shrinks by subtraction, not division. Use unrolling for those (6.1, 6.4, 6.5).

6.8 Recursion and space

Time and stack space are different questions. The stack space is the **depth** of the recursion tree: n for `factorial` and for the naive Fibonacci, log n for binary search and fast power, about log n for merge sort. See chapter 5.

7. Searching Algorithms

7.1 Linear search

Look at each element in turn until you find the target or run out of elements.

Listing 7.1

```
static int linearSearch(int[] a, int target) {
    for (int i = 0; i < a.length; i++) {
        if (a[i] == target) return i;     // found: stop early
    }
    return -1;                            // checked all n elements
}
```

Table 7.1

Case	Comparisons	Complexity
Best: target at index 0	1	O(1)
Average: target equally likely anywhere	(n + 1) / 2	O(n)
Worst: last element or absent	n	O(n)

Auxiliary space: O(1). It works on unsorted data, linked lists and streams. The measured step counts for a 5-element array (1, 3, 5 and 5) appear in Table 4.8.

7.2 Binary search

Precondition: the array is sorted. Compare the target with the middle element. If equal, done. If the target is bigger, everything left of the middle is too small, so discard the left half; otherwise discard the right half. Repeat on what remains.

Listing 7.2: iterative binary search

```
static int binarySearch(int[] a, int target) {      // a must be sorted ascending
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;     // avoids overflow of (lo + hi)
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1;   // discard left half
        else hi = mid - 1;                   // discard right half
    }
    return -1;
}
```

Table 7.2: searching 23 in {2, 5, 8, 12, 16, 23, 38, 56, 72, 91} (the program returns index 5)

Step	lo	hi	mid = lo + (hi − lo)/2	a[mid]	Decision
1	0	9	4	16	16 < 23: lo = mid + 1 = 5
2	5	9	7	56	56 > 23: hi = mid − 1 = 6
3	5	6	5	23	found, return 5

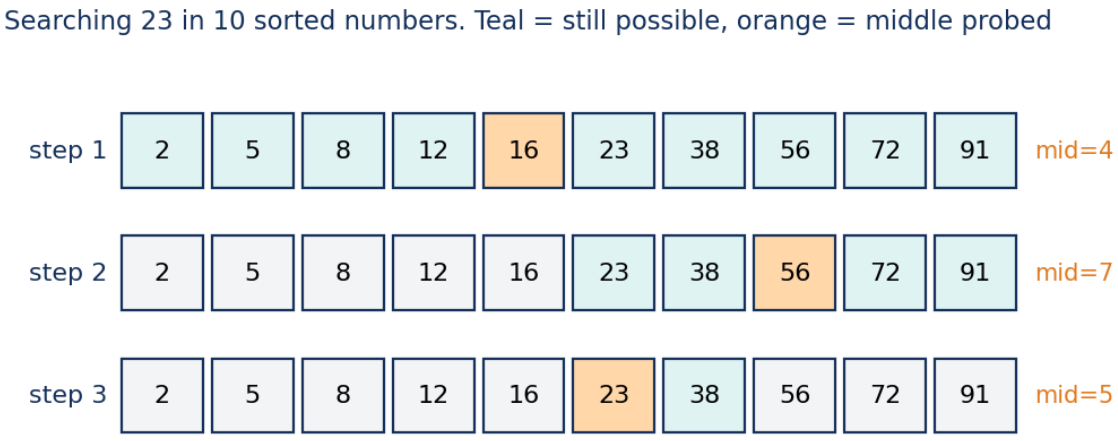


Figure 9. Each step discards half of the candidates (teal cells are still possible).

Searching for 40 (absent): the probes are a[4] = 16, a[7] = 56, a[5] = 23, a[6] = 38, then lo > hi and the method returns −1 (the program prints −1, verified).

Derivation 7.1. Why O(log n)? Let k be the number of loop iterations. The number of candidates is at most n, then ⌊n/2⌋, then ⌊n/4⌋, … Each iteration at least halves the candidates, and the loop stops when none remain. After j iterations at most $n/2^j$ candidates remain; this is below 1 when $2^j > n$, i.e. $j > \log_2 n$. So the loop runs at most $\lfloor \log_2 n \rfloor + 1$ times. (We confirmed by brute force over every array size from 1 to 299 and every possible target that the worst case is exactly $\lfloor \log_2 n \rfloor + 1$ iterations.)

Table 7.3: $\lfloor \log_2 n \rfloor + 1$ gives 4, 10, 20 and 30

n	Linear search (worst)	Binary search (worst iterations)
10	10	4
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

Listing 7.3: recursive binary search

```
static int binaryRec(int[] a, int target, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == target) return mid;
    if (a[mid] < target) return binaryRec(a, target, mid + 1, hi);
    return binaryRec(a, target, lo, mid - 1);         // depth <= log2 n + 1
}
```

Table 7.4

Version	Time	Auxiliary space
Iterative	O(log n)	O(1)
Recursive	O(log n)	O(log n) stack

Watch out. Three classic bugs: (1) `mid = (lo + hi) / 2` can overflow for huge arrays, so write `lo + (hi - lo) / 2`; (2) using `lo < hi` instead of `lo <= hi` skips the last candidate; (3) forgetting `+ 1` or `- 1` when moving the bounds causes an infinite loop.

Interview tip. If you need to search an unsorted array just once, linear search O(n) beats sorting first (O(n log n)). If you will search many times, sort once and use binary search for each query.

8. Sorting Algorithms

Six classic sorts, each with its Java code, a trace, a derivation of its comparison count, and its space and stability. All six were run against `Arrays.sort` on 3,000 random arrays each (lengths 0 to 39, including duplicates and negatives) and agreed every time.

> **Two terms.** **Stable** means equal keys keep their original relative order. **In place** means O(1) (or O(log n)) extra space.

8.1 Bubble sort

Analogy: bubbles rise. On each pass, compare neighbours and swap them if they are in the wrong order; the largest value drifts to the end.

Listing 8.1

```
static void bubbleSort(int[] a) {
    int n = a.length;
    for (int pass = 0; pass < n - 1; pass++) {
        boolean swapped = false;
        for (int j = 0; j < n - 1 - pass; j++) {
            if (a[j] > a[j + 1]) {        // compare neighbours
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;              // already sorted: stop early
    }
}
```

Table 8.1: bubble sort on [5, 2, 4, 1, 3] (starting array [5, 2, 4, 1, 3]); 10 comparisons, 7 swaps

Pass	Array after the pass	Swapped?
1	[2, 4, 1, 3, 5]	yes
2	[2, 1, 3, 4, 5]	yes
3	[1, 2, 3, 4, 5]	yes
4	[1, 2, 3, 4, 5]	no (stop)

> **Derivation 8.1. Comparisons.** Pass p (counting from 0) makes n − 1 − p comparisons. Total = (n−1) + (n−2) + … + 1 = n(n−1)/2 (Gauss). Worst case (reversed input): every comparison swaps, so n(n−1)/2 swaps as well. Best case (already sorted): the first pass makes n − 1 comparisons, finds no swap and stops: n − 1 comparisons. Average: each pair is out of order with probability ½, so about n(n−1)/4 swaps. Time: best O(n), average and worst O(n²). Space O(1). Stable (equal neighbours are never swapped).

8.2 Selection sort

Analogy: picking the shortest person from a line and moving them to the front, then repeating with the rest.

Listing 8.2

```
static void selectionSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min]) min = j;   // find the smallest in a[i..n-1]
        }
        int t = a[i]; a[i] = a[min]; a[min] = t;      // at most one swap per pass
    }
}
```

Table 8.2: selection sort on [5, 2, 4, 1, 3]; 10 comparisons

Pass i	Index of minimum found	Array after the swap
0	3 (value 1)	[1, 2, 4, 5, 3]
1	1 (value 2)	[1, 2, 4, 5, 3]
2	4 (value 3)	[1, 2, 3, 5, 4]
3	4 (value 4)	[1, 2, 3, 4, 5]

Derivation 8.2. Comparisons. Pass i scans positions i+1 … n−1, that is n − 1 − i comparisons, *whatever the data*. Total = (n−1) + … + 1 = n(n−1)/2: best, average and worst are all Θ(n²). It does at most n − 1 swaps, which is useful when writes are expensive. It is **not stable**: sorting [3a, 3b, 1] swaps 3a with 1 and produces [1, 3b, 3a], reversing the two 3s. Space O(1).

8.3 Insertion sort

Analogy: sorting playing cards in your hand. Pick up the next card and slide it left past every larger card until it fits.

Listing 8.3

```
static void insertionSort(int[] a) {
    for (int i = 1; i < a.length; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {    // shift larger elements one step right
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;                   // drop key into its gap
    }
}
```

Table 8.3: insertion sort on [5, 2, 4, 1, 3]; 9 comparisons in total

i	key	Comparisons	Array after inserting
1	2	1	[2, 5, 4, 1, 3]
2	4	2	[2, 4, 5, 1, 3]
3	1	3	[1, 2, 4, 5, 3]
4	3	3	[1, 2, 3, 4, 5]

Derivation 8.3. Best case (sorted): every key is compared once with its left neighbour and stays: n − 1 comparisons, O(n). **Worst case** (reversed): key i moves past all i earlier elements: 1 + 2 + … + (n−1) = n(n−1)/2 comparisons, O(n²). **Average case**: a random key is larger than about half of the earlier elements, so it shifts about i/2 places; Σ i/2 = n(n−1)/4, about n²/4 shifts. In general the number of shifts equals the number of *inversions* (pairs in the wrong order), so the time is Θ(n + inversions). Space O(1), stable. Insertion sort is the fastest simple sort on small or nearly sorted data.

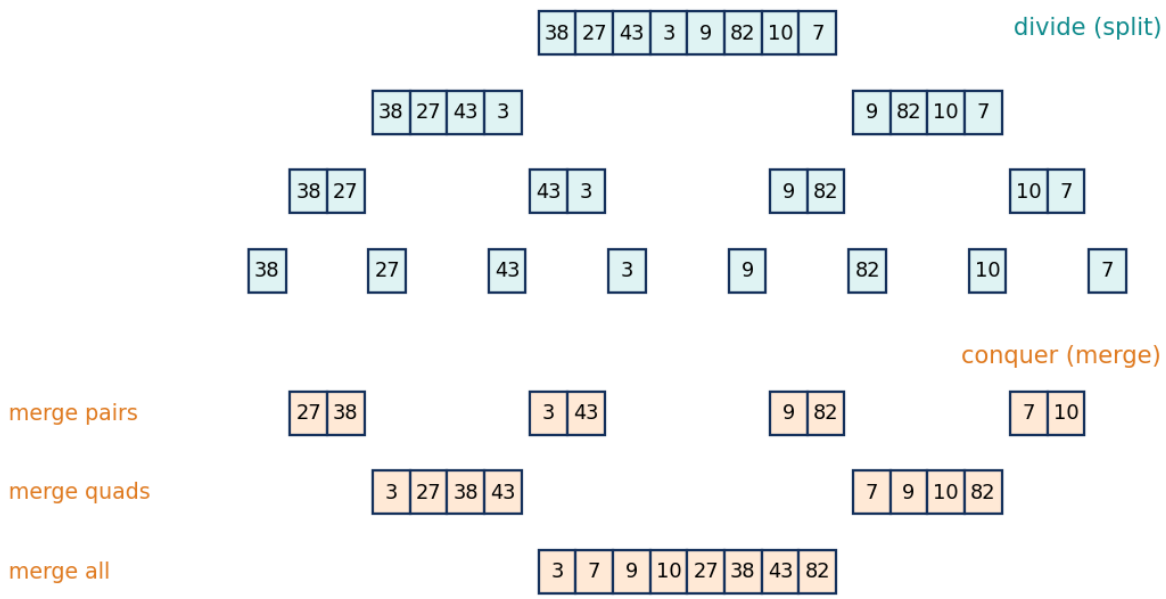
8.4 Merge sort

Analogy: divide a stack of exam papers into two halves, have each half sorted, then interleave the two sorted stacks.

Listing 8.4

```
static void mergeSort(int[] a, int lo, int hi) {   // sorts a[lo..hi-1]
    if (hi - lo < 2) return;
    int mid = (lo + hi) / 2;
    mergeSort(a, lo, mid);
    mergeSort(a, mid, hi);
    merge(a, lo, mid, hi);
}

static void merge(int[] a, int lo, int mid, int hi) {
    int[] tmp = new int[hi - lo];         // auxiliary array: O(n) extra space
    int i = lo, j = mid, k = 0;
    while (i < mid && j < hi) {
        if (a[i] <= a[j]) tmp[k++] = a[i++];   // '<=' keeps the sort stable
        else tmp[k++] = a[j++];
    }
    while (i < mid) tmp[k++] = a[i++];
    while (j < hi) tmp[k++] = a[j++];
    System.arraycopy(tmp, 0, a, lo, tmp.length);
}
```



3 levels of splitting ($\log_2 8 = 3$); each level of merging touches all 8 numbers → $8 \times 3 = 24$ steps

Figure 10. Merge sort on 8 numbers: 3 levels of splitting, 3 levels of merging.

Table 8.4: the 7 merges, in the order the code performs them

Merge (a[lo..hi−1])	Left run	Right run	Result
[0, 2)	[38]	[27]	[27, 38]
[2, 4)	[43]	[3]	[3, 43]
[0, 4)	[27, 38]	[3, 43]	[3, 27, 38, 43]
[4, 6)	[9]	[82]	[9, 82]
[6, 8)	[10]	[7]	[7, 10]
[4, 8)	[9, 82]	[7, 10]	[7, 9, 10, 82]
[0, 8)	[3, 27, 38, 43]	[7, 9, 10, 82]	[3, 7, 9, 10, 27, 38, 43, 82]

Derivation 8.4. Time. Merging two sorted runs of total length m takes at most m − 1 comparisons and m writes: O(m). The recurrence is T(n) = 2T(n/2) + n, solved in chapter 6 (Unrolling 6.3): Θ(n log n) in *every* case. Measured: for n = 1,000 random values, 8,712 comparisons; n $\log_2$ n ≈ 9,966. **Space.** The merge uses a temporary array of up to n elements: O(n); the recursion adds O(log n) frames; total O(n). **Stable** because `<=` takes from the left run when keys are equal.

8.5 Quick sort

Analogy: pick a pivot person; everyone shorter stands to the left, everyone taller to the right; the pivot is now in its final place. Repeat on each side.

Listing 8.5: Lomuto partition, pivot = last element

```
static void quickSort(int[] a, int lo, int hi) {   // sorts a[lo..hi]
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}

static int partition(int[] a, int lo, int hi) {    // Lomuto scheme, pivot = last element
    int pivot = a[hi];
    int i = lo;                            // a[lo..i-1] holds values < pivot
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            int t = a[i]; a[i] = a[j]; a[j] = t;
            i++;
        }
    }
    int t = a[i]; a[i] = a[hi]; a[hi] = t;         // put pivot in its final place
    return i;
}
```

Table 8.5: quick sort on [38, 27, 43, 3, 9, 82, 10], partitions in the order performed

Sub-array	Pivot	Pivot final index	Array after partition
[0..6]	10	2	[3, 9, 10, 38, 27, 82, 43]
[0..1]	9	1	[3, 9, 10, 38, 27, 82, 43]
[3..6]	43	5	[3, 9, 10, 38, 27, 43, 82]
[3..4]	27	3	[3, 9, 10, 27, 38, 43, 82]

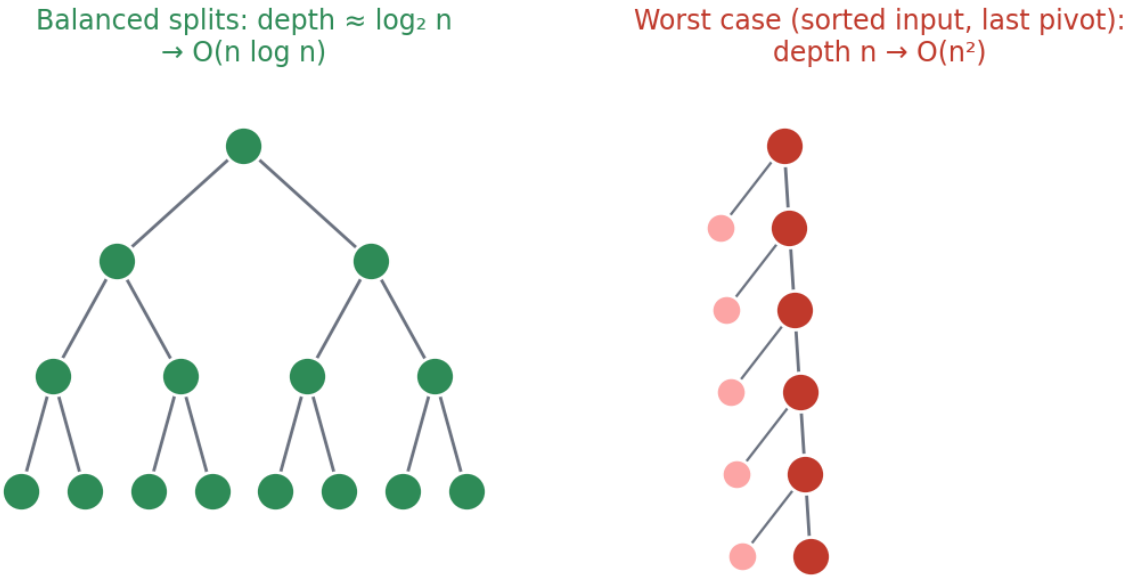


Figure 11. Balanced splits give depth log n; sorted input with a last-element pivot gives depth n.

Derivation 8.5. **Partitioning** a sub-array of size m makes m − 1 comparisons. **Best / average:** splits are balanced enough that T(n) = 2T(n/2) + n, so Θ(n log n); for random input the expected comparison count is $2(n+1)H_n - 4n \approx 1.39\, n \log_2 n$. Measured at n = 100: 644 (formula: ≈ 648). **Worst:** if the pivot is always the smallest or largest, T(n) = T(n−1) + (n−1) = n(n−1)/2 (Unrolling 6.5): Θ(n^2). With a last-element pivot this happens on sorted or reverse-sorted input: measured 4,950 comparisons at n = 100, which is exactly 100·99/2. **Fixes:** random pivot, median-of-three, and recursing into the smaller part first (keeps the stack at O(log n)). **Space:** O(log n) average stack, O(n) worst; in place; not stable.

8.6 Heap sort

Analogy: a tournament. Build a "max-heap" where every parent is at least as big as its children, so the champion is at the root. Repeatedly swap the champion to the back of the array and repair the heap.

Listing 8.6

```
static void heapSort(int[] a) {
    int n = a.length;
    for (int i = n / 2 - 1; i >= 0; i--) siftDown(a, i, n);   // build max-heap: O(n)
    for (int end = n - 1; end > 0; end--) {
        int t = a[0]; a[0] = a[end]; a[end] = t;     // move current max to the back
        siftDown(a, 0, end);                          // repair the heap: O(log n)
    }
}

static void siftDown(int[] a, int i, int size) {
    while (true) {
        int left = 2 * i + 1, right = 2 * i + 2, big = i;
        if (left < size && a[left] > a[big]) big = left;
        if (right < size && a[right] > a[big]) big = right;
        if (big == i) return;
        int t = a[i]; a[i] = a[big]; a[big] = t;
        i = big;
    }
}
```

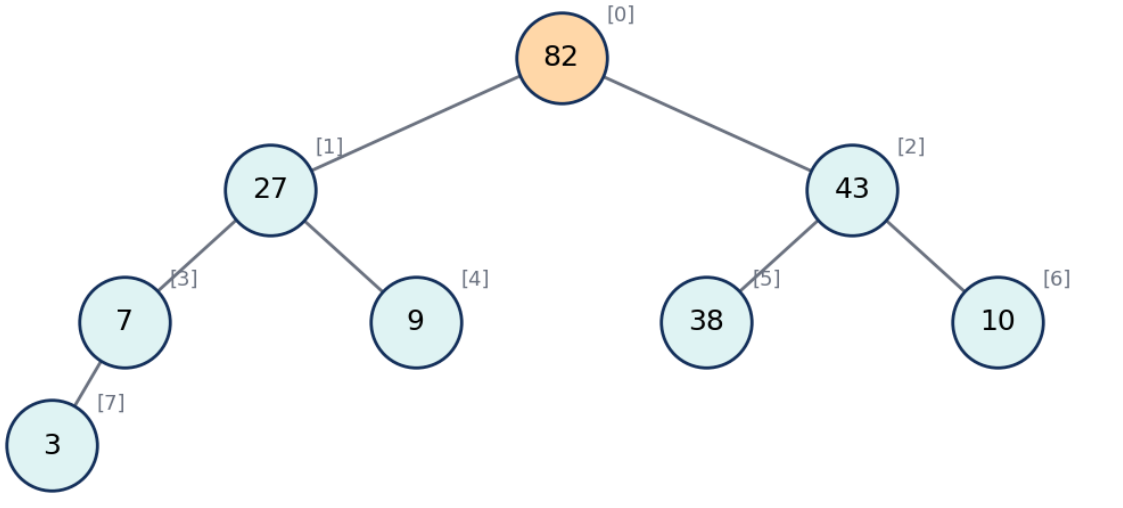


Figure 12. The max-heap built from [38, 27, 43, 3, 9, 82, 10, 7]. Index i has children 2i + 1 and 2i + 2.

Table 8.6: heap sort; the heap built first is [82, 27, 43, 7, 9, 38, 10, 3]

After moving max to index	Array (sorted suffix at the right)
7	[43, 27, 38, 7, 9, 3, 10, 82]
6	[38, 27, 10, 7, 9, 3, 43, 82]
5	[27, 9, 10, 7, 3, 38, 43, 82]
4	[10, 9, 3, 7, 27, 38, 43, 82]
3	[9, 7, 3, 10, 27, 38, 43, 82]
2	[7, 3, 9, 10, 27, 38, 43, 82]
1	[3, 7, 9, 10, 27, 38, 43, 82]

Derivation 8.6. Building the heap is O(n), not O(n log n). Nodes at height h (leaves have h = 0) number at most $\lceil n/2^{h+1} \rceil$, and sifting one costs at most h swaps. Total $\le \Sigma\, h \cdot n/2^{h+1} = (n/2) \cdot \Sigma\, h/2^h$. The series Σ for $h \ge 0$ of $h/2^h$ equals $x/(1-x)^2$ at $x = \frac{1}{2}$, which is $(\frac{1}{2})/(\frac{1}{4}) = 2$. So the total is $\le (n/2) \cdot 2 = n$. **Extraction:** $n - 1$ swaps, each followed by a sift-down of at most $\log_2 n$ levels: $O(n \log n)$. Total $\Theta(n \log n)$ in every case, space O(1), not stable.

8.7 Comparison of all six

Table 8.7

Algorithm	Best	Average	Worst	Aux. space	Stable	In place
Bubble (with flag)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	yes	yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	no	yes
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	yes	yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	yes	no
Quick (last pivot)	O(n log n)	O(n log n)	$O(n^2)$	O(log n) avg	no	yes
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	no	yes

Measured comparison counts, n = 100

Table 8.8: comparisons counted by instrumented copies of the sorts (each output verified sorted)

Algorithm	Sorted input	Reversed input	Random (mean of 20)
Bubble	99	4,950	4,874
Selection	4,950	4,950	4,950
Insertion	99	4,950	2,598
Merge	316	356	542
Quick	4,950	4,950	644
Heap	1,081	944	1,028

Read the table: bubble and insertion do only 99 comparisons on sorted input (O(n)), but 4,950 on reversed input ($O(n^2)$). Selection never cares. Quick sort with a last-element pivot is as bad as bubble sort on sorted input, yet fast on random data.

The doubling experiment

Double n and watch the ratio of comparisons. Quadratic algorithms grow about 4×; n log n algorithms a little over 2×.

Table 8.9: random input, mean of 5 runs

Algorithm	n = 1,000	n = 2,000	n = 4,000	ratio 2,000/1,000	ratio 4,000/2,000
Bubble	498,791	1,995,855	7,993,138	4.00	4.00
Selection	499,500	1,999,000	7,998,000	4.00	4.00
Insertion	251,182	1,002,547	4,047,681	3.99	4.04
Merge	8,712	19,424	42,833	2.23	2.21
Quick	10,458	24,415	55,390	2.33	2.27
Heap	16,842	37,709	83,426	2.24	2.21

In practice. Java's `Arrays.sort(int[])` uses a dual-pivot quick sort; `Arrays.sort(Object[])` and `Collections.sort` use TimSort (a stable merge/insertion hybrid, O(n log n) worst case). In an interview, say which sort you would use and why: "merge sort if I need stability or guaranteed O(n log n), heap sort for O(1) space with a guarantee, quick sort for speed on average, insertion sort for tiny or nearly sorted arrays."

9. Fifteen Practice Problems

Every problem below is original, written in the style of coding-interview platforms. For each one you get an example, a brute-force solution, an optimised solution, a trace of the optimised idea on the example, and the complexity of both. Every pair of solutions was run on the examples and compared on thousands of random inputs (see the verification note in chapter 15).

Table 9.1: overview

#	Problem	Technique	Brute force	Optimised
1	Pair With Target Sum	hash map	O(n²)	O(n)
2	First Non-Repeating Character	frequency array	O(n²)	O(n)
3	Best Window Sum	sliding window	O(n·k)	O(n)
4	Sorted Squares	two pointers	O(n log n)	O(n)
5	Missing Number	math formula	O(n²)	O(n)
6	Merge Two Sorted Arrays	two pointers	O((m+n) log(m+n))	O(m+n)
7	Rotate Array Right by k	three reversals	O(n·k)	O(n)
8	Count Subarrays With Sum k	prefix sums + map	O(n²)	O(n)
9	Longest Substring Without Repeats	sliding window	O(n²)	O(n)
10	Find a Peak	binary search	O(n)	O(log n)
11	Integer Square Root	binary search on answer	O(√x)	O(log x)
12	Stair Climb Ways	dynamic programming	O(2ⁿ)	O(n)
13	Product Except Self	prefix/suffix	O(n²)	O(n)
14	Search a Staircase Matrix	staircase walk	O(m·n)	O(m+n)
15	K-th Largest Element	min-heap	O(n log n)	O(n log k)

9.1 Problem 1: Pair With Target Sum

Level: Easy. **Statement.** Given an integer array `nums` and an integer `target`, return the indices `[i, j]` with `i < j` such that `nums[i] + nums[j] == target`. Exactly one such pair exists.

Input	Output	Explanation
`nums = [8, 3, 11, 5], target = 14`	`[1, 2]`	3 + 11 = 14.
`nums = [4, 4], target = 8`	`[0, 1]`	The same value may be used at two different indices.

Constraints: 2 ≤ n ≤ 10⁵; values fit in `int`.

Brute force

Try every pair (i, j) with i < j and return the first pair that adds to the target.

Brute force — Problem 1

```
static int[] pairSumBrute(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) return new int[]{i, j};
        }
    }
    return new int[]{-1, -1};
}
```

Complexity. The loops test n(n−1)/2 pairs in the worst case: O(n²) time. Extra space O(1).

Optimised

Scan left to right. For each `nums[j]` ask: "have I already seen the number `target − nums[j]`?" A hash map from value to index answers in O(1) on average. If yes, we are done; if not, remember `nums[j]`.

Table 9.2: optimised trace on nums = [8, 3, 11, 5], target = 14

Step j	nums[j]	Need = 14 − nums[j]	Map before (value→index)	Action
0	8	6	{ }	6 not found; store 8→0
1	3	11	{8→0}	11 not found; store 3→1
2	11	3	{8→0, 3→1}	3 found at index 1: return [1, 2]

Optimised — Problem 1

```
static int[] pairSumOpt(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();      // value -> index
    for (int j = 0; j < nums.length; j++) {
        Integer i = seen.get(target - nums[j]);        // O(1) average lookup
        if (i != null) return new int[]{i, j};
        seen.put(nums[j], j);
    }
    return new int[]{-1, -1};
}
```

Complexity. One pass of n steps, each doing an average O(1) hash lookup and insert: O(n) time. The map can hold n entries: O(n) space. (Worst-case hash behaviour is rarer and handled by Java's tree bins.)

> **Takeaway.** Trade space for time: a hash map turns the inner "search for the partner" loop into one lookup.

9.2 Problem 2: First Non-Repeating Character

Level: Easy. **Statement.** Given a string `s` of lowercase English letters, return the index of the first character that appears exactly once, or −1 if there is none.

Input	Output	Explanation
`"swiss"`	1	s appears 3 times, w once.
`"level"`	2	l and e repeat; v appears once.
`"aabb"`	−1	Every character repeats.

Constraints: 1 ≤ s.length ≤ 10^5.

Brute force

For each position i, scan the whole string to see whether the character occurs elsewhere.

Brute force — Problem 2

```
static int firstUniqueBrute(String s) {
    for (int i = 0; i < s.length(); i++) {
        boolean repeated = false;
        for (int j = 0; j < s.length(); j++) {
            if (i != j && s.charAt(i) == s.charAt(j)) { repeated = true; break; }
        }
        if (!repeated) return i;
    }
    return -1;
}
```

Complexity. For each of n positions we scan n characters: $O(n^2)$ time, O(1) space.

Optimised

Count every letter once into an array of 26 counters, then walk the string again and return the first index whose count is 1. Two linear passes.

Table 9.3: two-pass trace on "swiss"

Pass	Work	Result
1 (count)	read s, w, i, s, s	freq: s = 3, w = 1, i = 1
2 (find)	index 0 → s has count 3, skip	—
	index 1 → w has count 1	return 1

Optimised — Problem 2

```
static int firstUniqueOpt(String s) {
    int[] freq = new int[26];                          // fixed size: O(1) space
    for (int i = 0; i < s.length(); i++) freq[s.charAt(i) - 'a']++;
    for (int i = 0; i < s.length(); i++) {
        if (freq[s.charAt(i) - 'a'] == 1) return i;
    }
    return -1;
}
```

Complexity. Two passes of n steps: O(n) time. The array has 26 cells regardless of n: O(1) space.

> **Takeaway.** A fixed-size frequency table is O(1) space even though it feels like "extra memory".

9.3 Problem 3: Best Window Sum

Level: Easy. **Statement.** Given an integer array `nums` and an integer `k`, return the largest sum of any `k` consecutive elements.

Input	Output	Explanation
`nums = [2, 1, 5, 1, 3, 2], k = 3`	9	The window [5, 1, 3] sums to 9.
`nums = [4], k = 1`	4	Only one window.

Constraints: $1 \le k \le n \le 10^5$.

Brute force

For every starting index, add up its k numbers from scratch.

Brute force — Problem 3

```
static int bestWindowBrute(int[] nums, int k) {
    int best = Integer.MIN_VALUE;
    for (int start = 0; start + k <= nums.length; start++) {
        int sum = 0;
        for (int t = start; t < start + k; t++) sum += nums[t];   // re-adds k numbers
        best = Math.max(best, sum);
    }
    return best;
}
```

Complexity. There are $n - k + 1$ windows and each costs k additions: $(n - k + 1) \cdot k$. This is $O(n \cdot k)$, and it is $O(n^2)$ when $k \approx n/2$. Space O(1).

Optimised

Consecutive windows share k − 1 numbers. Compute the first window once, then slide: add the entering number, subtract the leaving one.

Table 9.4: sliding the window over [2, 1, 5, 1, 3, 2] with k = 3

i	Added nums[i]	Dropped nums[i−k]	Window sum	Best so far
init	—	—	8	8
3	1	2	7	8
4	3	1	9	9
5	2	5	6	9

Optimised — Problem 3

```
static int bestWindowOpt(int[] nums, int k) {
    int sum = 0;
    for (int i = 0; i < k; i++) sum += nums[i];        // first window
    int best = sum;
    for (int i = k; i < nums.length; i++) {
        sum += nums[i] - nums[i - k];                  // slide: add new, drop old
        best = Math.max(best, sum);
    }
    return best;
}
```

Complexity. The first window costs k; each of the remaining n − k slides costs 2 operations: k + 2(n − k) = O(n). Space O(1).

> **Takeaway.** Reuse overlapping work instead of recomputing it. This is the sliding-window pattern.

9.4 Problem 4: Sorted Squares

Level: Easy. **Statement.** Given an array `nums` sorted in non-decreasing order, return an array of the squares of each number, also in non-decreasing order.

Input	Output	Explanation
`[-4, -1, 0, 3, 10]`	`[0, 1, 9, 16, 100]`	Squares are 16, 1, 0, 9, 100; sorted gives the result.
`[-7, -3, 2, 3, 11]`	`[4, 9, 9, 49, 121]`	Negatives become positive.

Constraints: $1 \le n \le 10^5$; $|values| \le 10^4$.

Brute force

Square every element, then sort the result.

Brute force — Problem 4

```
static int[] sortedSquaresBrute(int[] nums) {
    int[] out = new int[nums.length];
    for (int i = 0; i < nums.length; i++) out[i] = nums[i] * nums[i];
    Arrays.sort(out);                                  // O(n log n)
    return out;
}
```

Complexity. Squaring is O(n); sorting is O(n log n): total O(n log n). It ignores that the input is already sorted.

Optimised

The largest square is at one of the two ends (the most negative or the most positive number). Keep two pointers, compare the squares at the ends, write the larger one at the back of the output, and move that pointer inward.

Table 9.5: filling the output from the back for [-4, -1, 0, 3, 10]

pos	left² (index)	right² (index)	Larger goes to out[pos]	out so far
4	16 (0)	100 (4)	100 (right)	[_, _, _, _, 100]
3	16 (0)	9 (3)	16 (left)	[_, _, _, 16, 100]
2	1 (1)	9 (3)	9 (right)	[_, _, 9, 16, 100]
1	1 (1)	0 (2)	1 (left)	[_, 1, 9, 16, 100]
0	0 (2)	0 (2)	0 (right)	[0, 1, 9, 16, 100]

Optimised — Problem 4

```
static int[] sortedSquaresOpt(int[] nums) {
    int n = nums.length;
    int[] out = new int[n];
    int left = 0, right = n - 1;
    for (int pos = n - 1; pos >= 0; pos--) {          // fill from the largest square
        int l = nums[left] * nums[left];
        int r = nums[right] * nums[right];
        if (l > r) { out[pos] = l; left++; }
        else       { out[pos] = r; right--; }
    }
    return out;
}
```

Complexity. Each of the n steps moves one pointer: O(n) time. The output array is O(n); extra space beyond the output is O(1).

> **Takeaway.** Use the structure of the input (sorted, with a "valley" shape of squares) to beat a generic sort.

9.5 Problem 5: Missing Number

Level: Easy. **Statement.** An array `nums` contains `n` distinct integers taken from the range `0..n`. Exactly one number of that range is missing; return it.

Input	Output	Explanation
`[3, 0, 1]`	2	Range is 0..3; 2 is absent.
`[0, 1, 2, 4]`	3	Range is 0..4; 3 is absent.

Constraints: $1 \le n \le 10^5$.

Brute force

For each candidate v from 0 to n, scan the array to see whether v appears.

Brute force — Problem 5

```
static int missingBrute(int[] nums) {
    int n = nums.length;
    for (int v = 0; v <= n; v++) {
        boolean found = false;
        for (int x : nums) if (x == v) { found = true; break; }
        if (!found) return v;
    }
    return -1;
}
```

Complexity. n + 1 candidates × up to n comparisons: O(n²). Space O(1).

Optimised

The sum of 0..n is n(n+1)/2. Subtract the sum of the array; the difference is the missing number. A `long` avoids overflow in the product.

Table 9.6: nums = [3, 0, 1]

n	Expected sum n(n+1)/2	Actual sum	Missing = expected − actual
3	3·4/2 = 6	3 + 0 + 1 = 4	6 − 4 = 2

Optimised — Problem 5

```
static int missingOpt(int[] nums) {
    long n = nums.length;
    long expected = n * (n + 1) / 2;                  // 0 + 1 + ... + n
    long actual = 0;
    for (int x : nums) actual += x;
    return (int) (expected - actual);
}
```

Complexity. One pass to sum: O(n) time; a few variables: O(1) space.

> **Takeaway.** A mathematical identity can replace a search entirely. (An XOR of all indices and values is an equivalent trick that cannot overflow.)

9.6 Problem 6: Merge Two Sorted Arrays

Level: Easy. **Statement.** Given two arrays `a` and `b`, each sorted in non-decreasing order, return one new sorted array containing all their elements.

Input	Output	Explanation
`a = [1, 4, 9], b = [2, 3, 10, 12]`	`[1, 2, 3, 4, 9, 10, 12]`	Interleave in order.
`a = [], b = [5]`	`[5]`	Empty inputs are allowed.

Constraints: 0 ≤ a.length, b.length ≤ 10⁵.

Brute force

Copy both arrays into one and sort it.

Brute force — Problem 6

```
static int[] mergeSortedBrute(int[] a, int[] b) {
    int[] out = new int[a.length + b.length];
    System.arraycopy(a, 0, out, 0, a.length);
    System.arraycopy(b, 0, out, a.length, b.length);
    Arrays.sort(out);                                 // ignores that inputs are sorted
    return out;
}
```

Complexity. Let N = m + n. Sorting costs O(N log N). Output O(N).

Optimised

This is the merge step of merge sort. Keep one pointer in each array, always copy the smaller front element, then copy whatever remains.

Table 9.7: merging a and b

Compare	Take	out so far
a[0]=1 vs b[0]=2	1 from a	[1]
a[1]=4 vs b[0]=2	2 from b	[1, 2]
a[1]=4 vs b[1]=3	3 from b	[1, 2, 3]
a[1]=4 vs b[2]=10	4 from a	[1, 2, 3, 4]
a[2]=9 vs b[2]=10	9 from a	[1, 2, 3, 4, 9]
a exhausted; copy the rest of b	10, 12	[1, 2, 3, 4, 9, 10, 12]

Optimised — Problem 6

```
static int[] mergeSortedOpt(int[] a, int[] b) {
    int[] out = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) out[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < a.length) out[k++] = a[i++];
    while (j < b.length) out[k++] = b[j++];
    return out;
}
```

Complexity. Every comparison writes one output element and N elements are written: O(m + n) time. Output space O(m + n); no other extra space.

> **Takeaway.** If the input already has order, exploit it: merging is linear while sorting from scratch is n log n.

9.7 Problem 7: Rotate Array Right by k

Level: Medium. **Statement.** Rotate `nums` to the right by `k` positions **in place**.

Input	Output	Explanation
`nums = [1, 2, 3, 4, 5, 6, 7], k = 3`	`[5, 6, 7, 1, 2, 3, 4]`	The last three elements move to the front.
`nums = [1, 2], k = 5`	`[2, 1]`	k is reduced modulo n: 5 mod 2 = 1.

Constraints: $1 \le n \le 10^5$; $k \ge 0$.

Brute force

Rotate by one position, k times: save the last element, shift everything right, put it in front.

Brute force — Problem 7

```
static void rotateBrute(int[] nums, int k) {
    int n = nums.length;
    k = k % n;
    for (int step = 0; step < k; step++) {            // k single-step rotations
        int last = nums[n - 1];
        for (int i = n - 1; i > 0; i--) nums[i] = nums[i - 1];
        nums[0] = last;
    }
}
```

Complexity. Each single rotation costs n; there are k % n of them: O(n·k) time (up to $O(n^2)$), O(1) space.

Optimised

Reversing the whole array puts the last k elements in front but backwards. Reverse the first k to fix their order, then reverse the remaining n − k.

Table 9.8: three reversals on [1, 2, 3, 4, 5, 6, 7], k = 3

Step	Operation	Array
0	start	[1, 2, 3, 4, 5, 6, 7]
1	reverse all	[7, 6, 5, 4, 3, 2, 1]
2	reverse first k = 3	[5, 6, 7, 4, 3, 2, 1]
3	reverse last n − k = 4	[5, 6, 7, 1, 2, 3, 4]

Optimised — Problem 7

```
static void rotateOpt(int[] nums, int k) {
    int n = nums.length;
    k = k % n;
    reverse(nums, 0, n - 1);                          // whole array
    reverse(nums, 0, k - 1);                          // first k
    reverse(nums, k, n - 1);                          // remaining n-k
}

static void reverse(int[] a, int i, int j) {
    while (i < j) { int t = a[i]; a[i] = a[j]; a[j] = t; i++; j--; }
}
```

Complexity. Each element is swapped at most once per reversal and the three reversals cover n + k + (n − k) = 2n elements in total (reversing a segment of length L performs ⌊L/2⌋ swaps, so at most n swaps overall): O(n) time and O(1) space.

> **Takeaway.** A composition of simple in-place operations can replace repeated shifting.

9.8 Problem 8: Count Subarrays With Sum k

Level: Medium. **Statement.** Given an integer array `nums` (it may contain negatives) and an integer `k`, return how many contiguous, non-empty subarrays sum to `k`.

Input	Output	Explanation
`nums = [1, 2, 3], k = 3`	2	[1, 2] and [3].
`nums = [1, 1, 1], k = 2`	2	Two overlapping windows of size 2.
`nums = [3, -1, 2, 1, -2, 4], k = 3`	4	[3], [3,-1,2,1,-2], [2,1], [1,-2,4].

Constraints: $1 \le n \le 10^5$; negatives allowed, so a sliding window does not work.

Brute force

Fix a start index, extend the end index while keeping a running sum, and count each time it equals k. (A naive triple loop that re-adds every subarray would be $O(n^3)$; the running sum removes the innermost loop.)

Brute force — Problem 8

```
static int countSubarraysBrute(int[] nums, int k) {
    int count = 0;
    for (int start = 0; start < nums.length; start++) {
        int sum = 0;
        for (int end = start; end < nums.length; end++) {
            sum += nums[end];                         // running sum of nums[start..end]
            if (sum == k) count++;
        }
    }
    return count;
}
```

Complexity. n(n+1)/2 (start, end) pairs: O(n²) time, O(1) space.

Optimised

Let P(i) be the sum of the first i numbers. The subarray (j, i] has sum P(i) − P(j). It equals k exactly when P(j) = P(i) − k. So while scanning, count how many earlier prefix sums equal P(i) − k, using a hash map of prefix-sum frequencies.

Table 9.9: nums = [1, 2, 3], k = 3 (the map starts as {0→1})

x	prefix	Look up prefix − k	Found count	Total	Map after
1	1	-2	0	0	{0→1, 1→1}
2	3	0	1	1	{0→1, 1→1, 3→1}
3	6	3	1	2	{0→1, 1→1, 3→1, 6→1}

Optimised — Problem 8

```
static int countSubarraysOpt(int[] nums, int k) {
    Map<Integer, Integer> seen = new HashMap<>();      // prefix sum -> how many times
    seen.put(0, 1);                                    // empty prefix
    int prefix = 0, count = 0;
    for (int x : nums) {
        prefix += x;
        count += seen.getOrDefault(prefix - k, 0);     // earlier prefixes that leave sum k
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}
```

Complexity. One pass with O(1) average map operations: O(n) time, O(n) space for up to n + 1 distinct prefix sums.

> **Takeaway.** Prefix sums turn "sum of a range" into a difference of two numbers, and a hash map turns "find a matching earlier prefix" into O(1).

9.9 Problem 9: Longest Substring Without Repeats

Level: Medium. **Statement.** Return the length of the longest substring of `s` in which all characters are distinct.

Input	Output	Explanation
`"abcabcbb"`	3	"abc".
`"pwwkew"`	3	"wke" (not "pwke": it must be contiguous).
`"bbbb"`	1	"b".

Constraints: 0 ≤ s.length ≤ 10⁵, ASCII characters.

Brute force

For each start index, extend the end while the next character has not yet been used in this window.

Brute force — Problem 9

```
static int longestUniqueBrute(String s) {
    int best = 0;
    for (int start = 0; start < s.length(); start++) {
        boolean[] used = new boolean[128];
        int len = 0;
        for (int end = start; end < s.length(); end++) {
            char c = s.charAt(end);
            if (used[c]) break;                        // repeat found: stop extending
            used[c] = true;
            len++;
        }
        best = Math.max(best, len);
    }
    return best;
}
```

Complexity. In the worst case (all characters distinct, within the 128-character alphabet) each start scans up to min(n, 128) characters, but with a larger alphabet it is O(n²). Space: O(1) (128 flags, re-allocated for each start).

Optimised

Slide a window [left, right]. Remember for each character the position just after its last occurrence. When the new character already lies inside the window, jump `left` past that earlier copy. `left` never moves backwards.

Table 9.10: window moves on "abcabcbb"

right	char	left after jump	Window	Length	Best
0	a	0	a	1	1
1	b	0	ab	2	2
2	c	0	abc	3	3
3	a	1	bca	3	3
4	b	2	cab	3	3
5	c	3	abc	3	3
6	b	5	cb	2	3
7	b	7	b	1	3

Optimised — Problem 9

```
static int longestUniqueOpt(String s) {
    int[] last = new int[128];                         // last index + 1 where char was seen
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        left = Math.max(left, last[c]);                // jump left past the previous copy
        best = Math.max(best, right - left + 1);
        last[c] = right + 1;
    }
    return best;
}
```

Complexity. `right` moves n times and `left` only moves forward, so every character is handled O(1) times: O(n) time. The array has 128 cells: O(1) space.

> **Takeaway.** Two pointers that both move only forward give a linear-time scan of all windows.

9.10 Problem 10: Find a Peak

Level: Medium. **Statement.** An element is a *peak* if it is strictly greater than its neighbours; treat positions outside the array as $-\infty$. Given `nums` in which adjacent elements are different, return the index of any peak.

Input	Output	Explanation
`[1, 3, 2, 4, 1]`	`1 or 3`	Both 3 (index 1) and 4 (index 3) are peaks. Our brute force returns 1, the optimised version returns 3; either is accepted.
`[1, 2, 3, 4, 5]`	`4`	Strictly increasing: the last element is a peak.

Constraints: $1 \le n \le 10^5$; nums[i] ≠ nums[i + 1].

Brute force

Check every index against its neighbours and return the first peak.

Brute force — Problem 10

```
static int findPeakBrute(int[] nums) {
    for (int i = 0; i < nums.length; i++) {
        boolean leftOk = (i == 0) || nums[i - 1] < nums[i];
        boolean rightOk = (i == nums.length - 1) || nums[i + 1] < nums[i];
        if (leftOk && rightOk) return i;
    }
    return -1;
}
```

Complexity. At most n checks of constant cost: O(n) time, O(1) space.

Optimised

Look at the middle and its right neighbour. If `nums[mid] < nums[mid+1]` we are going uphill, so a peak must exist to the right (the array eventually ends in $-\infty$). Otherwise a peak exists at `mid` or to the left. Keep the half that is guaranteed to contain a peak.

Table 9.11: nums = [1, 3, 2, 4, 1]

lo	hi	mid	nums[mid] vs nums[mid+1]	Action
0	4	2	2 < 4	uphill: lo = 3
3	4	3	4 > 1	downhill: hi = 3
3	3	—	lo = hi	return 3

Optimised — Problem 10

```
static int findPeakOpt(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] < nums[mid + 1]) lo = mid + 1;   // uphill: a peak lies to the right
        else hi = mid;                                  // downhill: a peak is at mid or left
    }
    return lo;
}
```

Complexity. Each iteration halves the range [lo, hi]: $\lfloor \log_2 n \rfloor + 1$ iterations at most, O(log n) time, O(1) space.

> **Takeaway.** Binary search does not need a fully sorted array; it needs a rule that tells you which half must contain an answer.

9.11 Problem 11: Integer Square Root

Level: Easy. **Statement.** Given an integer `x ≥ 0`, return ⌊√x⌋ without using a built-in square root.

Input	Output	Explanation
17	4	$4^2 = 16 \le 17 < 25 = 5^2$.
25	5	Perfect square.
0	0	Edge case.

Constraints: $0 \le x \le 2^{31} - 1$.

Brute force

Increase i until $(i + 1)^2$ exceeds x.

Brute force — Problem 11

```
static int sqrtBrute(int x) {
    long i = 0;
    while ((i + 1) * (i + 1) <= x) i++;                // about sqrt(x) steps
    return (int) i;
}
```

Complexity. The loop runs about √x times: O(√x). For x near $2 \cdot 10^9$ that is about 46,000 steps; fine here, but exponentially worse than needed in terms of the number of bits of x. Space O(1).

Optimised

The answer lies in [0, x] and "$mid^2 \le x$" is true for small mid and false for large mid. Binary-search the largest mid for which it is true. Use `long` so mid·mid cannot overflow.

Table 9.12: x = 17

lo	hi	mid	mid^2	Compare with 17	Action
0	17	8	64	> 17	hi = 7
0	7	3	9	≤ 17	ans = 3; lo = 4
4	7	5	25	> 17	hi = 4
4	4	4	16	≤ 17	ans = 4; lo = 5

Optimised — Problem 11

```
static int sqrtOpt(int x) {
    long lo = 0, hi = x, ans = 0;
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2;
        if (mid * mid <= x) { ans = mid; lo = mid + 1; }   // mid works, try bigger
        else hi = mid - 1;
    }
    return (int) ans;
}
```

Complexity. The search range [0, x] halves each time: O(log x) iterations. Space O(1). (Tested against `Math.sqrt` for every x up to 5,000 and for the largest values of int.)

> **Takeaway.** "Binary search on the answer" works whenever a yes/no test is monotone: false … false true … true.

9.12 Problem 12: Stair Climb Ways

Level: Easy. **Statement.** You climb a staircase of `n` steps, taking 1 or 2 steps at a time. How many distinct sequences of moves reach the top?

Input	Output	Explanation
`n = 5`	8	One sequence of all 1s, four sequences with a single 2, three with two 2s: 1 + 4 + 3 = 8.
`n = 10`	89	Follows the Fibonacci pattern.

Constraints: $0 \le n \le 45$ (the answer fits in `int`).

Brute force

The last move is a 1-step (leaving n−1 steps) or a 2-step (leaving n−2): ways(n) = ways(n−1) + ways(n−2). Code it directly.

Brute force — Problem 12

```
static int waysBrute(int n) {
    if (n <= 1) return 1;
    return waysBrute(n - 1) + waysBrute(n - 2);        // recomputes the same values again and again
}
```

Complexity. This is the same recursion as naive Fibonacci (Derivation 3.2): $O(2^n)$ time (more precisely about 1.618^n), O(n) stack space.

Optimised

ways(n) depends only on the previous two values. Keep just those two in variables and iterate upward.

Table 9.13: building up the answer

i	0	1	2	3	4	5
ways(i)	1	1	2	3	5	8

Optimised — Problem 12

```
static int waysOpt(int n) {
    int prev = 1, cur = 1;                              // ways(0), ways(1)
    for (int i = 2; i <= n; i++) {
        int next = prev + cur;
        prev = cur;
        cur = next;
    }
    return cur;
}
```

Complexity. One loop of n − 1 steps: O(n) time, O(1) space.

> **Takeaway.** Overlapping subproblems → dynamic programming. Keeping only the values you still need reduces O(n) space to O(1).

9.13 Problem 13: Product Except Self

Level: Medium. **Statement.** Given `nums`, return an array `out` where `out[i]` is the product of all elements except `nums[i]`. Do not use division. Assume the products fit in `int`.

Input	Output	Explanation
`[1, 2, 3, 4]`	`[24, 12, 8, 6]`	out[0] = 2·3·4 = 24, etc.
`[2, 0, 5]`	`[0, 10, 0]`	Zeros are handled correctly (no division).

Constraints: $2 \le n \le 10^5$.

Brute force

For every i, multiply all the other elements.

Brute force — Problem 13

```
static int[] productExceptSelfBrute(int[] nums) {
    int n = nums.length;
    int[] out = new int[n];
    for (int i = 0; i < n; i++) {
        int prod = 1;
        for (int j = 0; j < n; j++) if (j != i) prod *= nums[j];
        out[i] = prod;
    }
    return out;
}
```

Complexity. n outer iterations × n inner multiplications: O(n²) time. O(1) extra space apart from the output.

Optimised

out[i] = (product of everything left of i) × (product of everything right of i). A forward pass stores left-products in `out`; a backward pass multiplies in a running right-product.

Table 9.14: nums = [1, 2, 3, 4]

Pass	Step	Running value	out
forward	i = 1..3	out[i] = out[i−1]·nums[i−1]	[1, 1, 2, 6]
backward	i = 3, right = 1	out[3] = 6·1 = 6; right = 4	[1, 1, 2, 6]
backward	i = 2	out[2] = 2·4 = 8; right = 12	[1, 1, 8, 6]
backward	i = 1	out[1] = 1·12 = 12; right = 24	[1, 12, 8, 6]
backward	i = 0	out[0] = 1·24 = 24	[24, 12, 8, 6]

Optimised — Problem 13

```
static int[] productExceptSelfOpt(int[] nums) {
    int n = nums.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * nums[i - 1];   // product of everything left of i
    int right = 1;
    for (int i = n - 1; i >= 0; i--) {
        out[i] *= right;                                  // multiply by product of everything right of i
        right *= nums[i];
    }
    return out;
}
```

Complexity. Two passes of n steps: O(n) time; O(1) auxiliary space (the output array is not counted).

> **Takeaway.** Precompute prefix and suffix information to avoid repeated work.

9.14 Problem 14: Search a Staircase Matrix

Level: Medium. **Statement.** An m × n matrix has rows sorted left to right and columns sorted top to bottom. Return `true` if `target` is present.

Input	Output	Explanation
`matrix = [[1,4,7,11],[2,5,8,12],[3,6,9,16],[10,13,14,17]], target = 9`	`true`	9 is at row 2, column 2.
`same matrix, target = 15`	`false`	15 does not appear.

Constraints: 1 ≤ m, n ≤ 1000.

Brute force

Check every cell.

Brute force — Problem 14

```
static boolean searchMatrixBrute(int[][] m, int target) {
    for (int[] row : m)
        for (int v : row)
            if (v == target) return true;
    return false;
}
```

Complexity. m·n cells: O(m·n) time, O(1) space.

Optimised

Start at the top-right corner. If the cell is bigger than the target, everything below it in its column is bigger too, so move left. If it is smaller, everything to its left in its row is smaller, so move down. Each step removes a row or a column.

Table 9.15: target = 9

Step	Cell (row, col)	Value	Compare with 9	Move
1	(0, 3)	11	>	left
2	(0, 2)	7	<	down
3	(1, 2)	8	<	down
4	(2, 2)	9	=	found, return true

Optimised — Problem 14

```
static boolean searchMatrixOpt(int[][] m, int target) {
    int r = 0, c = m[0].length - 1;                    // start at the top-right corner
    while (r < m.length && c >= 0) {
        int v = m[r][c];
        if (v == target) return true;
        if (v > target) c--;                           // everything below in this column is even bigger
        else r++;                                      // everything left in this row is even smaller
    }
    return false;
}
```

Complexity. Each step eliminates one row or one column, so at most m + n steps: O(m + n) time, O(1) space.

> **Takeaway.** Choose a starting point where both directions give a clear decision.

9.15 Problem 15: K-th Largest Element

Level: Medium. **Statement.** Return the k-th largest element of an unsorted array (counting duplicates: in [3, 3, 3, 1] the 3rd largest is 3).

Input	Output	Explanation
`nums = [4, 9, 1, 7, 5], k = 2`	7	Sorted descending: 9, 7, 5, 4, 1.
`nums = [3, 3, 3, 1], k = 3`	3	Duplicates count.

Constraints: 1 ≤ k ≤ n ≤ 10^5.

Brute force

Sort a copy and index from the end.

Brute force — Problem 15

```
static int kthLargestBrute(int[] nums, int k) {
    int[] copy = nums.clone();
    Arrays.sort(copy);                                  // ascending
    return copy[copy.length - k];
}
```

Complexity. Sorting costs O(n log n); the copy needs O(n) extra space.

Optimised

Keep a min-heap of the k largest numbers seen so far. Insert each number; if the heap grows past k, remove its minimum. At the end the heap's minimum is the k-th largest.

Table 9.16: nums = [4, 9, 1, 7, 5], k = 2

Number	Heap after insert	Size > 2? remove min	Heap after
4	{4}	no	{4}
9	{4, 9}	no	{4, 9}
1	{1, 4, 9}	yes: remove 1	{4, 9}
7	{4, 7, 9}	yes: remove 4	{7, 9}
5	{5, 7, 9}	yes: remove 5	{7, 9}

Optimised — Problem 15

```
static int kthLargestOpt(int[] nums, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();   // min-heap holding the k largest so far
    for (int x : nums) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();              // drop the smallest of the k+1
    }
    return heap.peek();                                // smallest of the k largest = k-th largest
}
```

Complexity. Each of the n numbers costs one insert and at most one removal on a heap of size ≤ k + 1, each O(log k): O(n log k) time, O(k) space. (Quickselect achieves O(n) average time with O(1) extra space.)

> **Takeaway.** When k is small, a bounded heap beats a full sort.

10. Thirty Multiple-Choice Questions

Ten beginner, ten intermediate and ten interview-level questions. Each has four options and exactly one correct answer. Work them on paper first; the answer key and a full explanation for every question are in chapter 15. For counting questions, trace the loop before you choose.

> **Note.** These are original practice questions. They are not claimed to be actual Google, Microsoft or LeetCode questions.

10.1 Beginner level (questions 1–10)

Q1. What is the time complexity of the statement `int x = a[5];` where `a` is an array of length n ≥ 6?

- **A.** O(n)
- **B.** O(1)
- **C.** O(log n)
- **D.** O(n²)

Q2. Simplify the step count T(n) = 4n² + 10n + 7 to Big-O.

- **A.** O(n)
- **B.** O(n³)
- **C.** O(4n²)
- **D.** O(n²)

Q3. Which description best fits this loop?

```
for (int i = 0; i < n; i += 2) {
    count++;
}
```

- **A.** O(n)
- **B.** O(log n)
- **C.** O(n²)
- **D.** O(√n)

Q4. How many times does `count++` run for n = 64?

```
for (int i = 1; i < n; i *= 2) {
    count++;
}
```

- **A.** 5
- **B.** 7
- **C.** 6
- **D.** 64

Q5. Two loops run one after the other, each from 0 to n − 1 with O(1) work in the body. The total time is...

- **A.** O(n²)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(1)

Q6. How many times does `count++` run when n = 10?

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
        count++;
```

- **A.** 100
- **B.** 10
- **C.** 20
- **D.** 55

Q7. As n becomes very large, which function grows fastest?

- **A.** n^3
- **B.** n^2
- **C.** n log n
- **D.** 2^n

Q8. What is the worst-case time complexity of linear search in an array of n elements?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** $O(n^2)$

Q9. What is the auxiliary space of a loop that sums the elements of an array using one `int sum` variable?

- **A.** O(1)
- **B.** O(n)
- **C.** O(log n)
- **D.** $O(n^2)$

Q10. Which precondition must hold for binary search to work?

- **A.** The array has no duplicates
- **B.** The array is sorted
- **C.** The array length is a power of two
- **D.** The array contains only positive numbers

10.2 Intermediate level (questions 11–20)

Q11. How many times does `count++` run for n = 10?

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        count++;
```

- **A.** 55
- **B.** 100
- **C.** 45
- **D.** 50

Q12. How many times does `count++` run for n = 16?

```
for (int i = 0; i < n; i++)
    for (int j = 1; j < n; j *= 2)
        count++;
```

- **A.** 64
- **B.** 16
- **C.** 256
- **D.** 80

Q13. What is the time complexity of this code?

```
for (int i = n; i > 0; i /= 2)
    for (int j = 0; j < i; j++)
        count++;
```

- **A.** O(n log n)
- **B.** O(log n)
- **C.** O(n²)
- **D.** O(n)

Q14. Solve the recurrence T(n) = 2T(n/2) + n.

- **A.** O(n)
- **B.** O(n log n)
- **C.** O(n²)
- **D.** O(log n)

Q15. What is the auxiliary (stack) space of the recursive method `int fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }`?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n)
- **D.** O(n²)

Q16. What is the best-case time complexity of insertion sort?

- **A.** O(1)
- **B.** O(log n)
- **C.** O(n²)
- **D.** O(n)

Q17. Which sorting algorithm below is Θ(n log n) on every input?

- **A.** Merge sort
- **B.** Quick sort with the last element as pivot
- **C.** Bubble sort with an early-exit flag
- **D.** Insertion sort

Q18. How many comparisons does selection sort perform on any array of n = 8 elements?

- **A.** 7
- **B.** 28
- **C.** 36
- **D.** 56

Q19. What is the time complexity of this code?

```
for (int i = 0; i < n; i++)
    for (int j = 0; j < 5; j++)
        count++;
```

- **A.** O(n²)
- **B.** O(n log n)
- **C.** O(log n)
- **D.** O(n)

Q20. Finding whether an array has a duplicate by inserting elements into a `HashSet` takes (average case)...

- **A.** O(n²) time and O(1) space
- **B.** O(n log n) time and O(1) space
- **C.** O(n) time and O(n) space
- **D.** O(n) time and O(1) space

10.3 Interview level (questions 21–30)

Q21. How many calls does the naive recursive `fib(10)` make in total (counting the first call)?

- **A.** 177
- **B.** 55
- **C.** 89
- **D.** 1,024

Q22. What is the time complexity of this code?

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j += i)
        count++;
```

- **A.** O(n²)
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(n√n)

Q23. What is the time complexity of this code?

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i * i; j++)
        if (j % i == 0)
            for (int k = 0; k < j; k++)
                count++;
```

- **A.** O(n³)
- **B.** O(n⁴)
- **C.** O(n⁵)
- **D.** O(n²)

Q24. Merging two sorted arrays of lengths m and n into a new array takes...

- **A.** O(m · n)
- **B.** O(min(m, n))
- **C.** O((m + n) log(m + n))
- **D.** O(m + n)

Q25. What is the worst-case time of quick sort that always picks the last element as pivot, on an already-sorted array?

- **A.** $O(n^2)$
- **B.** O(n log n)
- **C.** O(n)
- **D.** O(log n)

Q26. What is the auxiliary space of top-down merge sort on an array?

- **A.** O(1)
- **B.** O(n)
- **C.** O(log n)
- **D.** O(n log n)

Q27. What is the time to build a binary max-heap from n unsorted elements using sift-down?

- **A.** O(n log n)
- **B.** O(log n)
- **C.** O(n)
- **D.** $O(n^2)$

Q28. A method solves a problem of size n by calling itself twice on size n/2 and doing O(1) other work. Its time complexity is...

- **A.** O(log n)
- **B.** O(n log n)
- **C.** $O(2^n)$
- **D.** O(n)

Q29. What is the amortized cost of `ArrayList.add(x)` (appending at the end)?

- **A.** O(n)
- **B.** O(1)
- **C.** O(log n)
- **D.** O(n log n)

Q30. What is the time complexity of building a string with `String s = ""; for (int i = 0; i < n; i++) s += "a";`?

- **A.** $O(n^2)$
- **B.** O(n)
- **C.** O(n log n)
- **D.** O(1)

11. Twenty Interview Questions

Practice questions on complexity, in the style asked in internship and entry-level interviews. Each answer is written the way you could say it aloud. They are original practice questions, not claimed actual questions from any company.

Q1. What is the difference between time complexity and space complexity?

Time complexity describes how the number of basic steps grows with the input size n. Space complexity describes how the extra memory used grows with n. Usually we report **auxiliary** space (extra memory, including the recursion stack, excluding the input). Example: merge sort is O(n log n) time and O(n) auxiliary space; heap sort is O(n log n) time and O(1) space.

Q2. Why do we drop constants and lower-order terms? When can this mislead?

Big-O describes growth for large n, where the fastest-growing term dominates and constant factors only shift the curve. $5n^2 + 100n$ and n^2 are both $O(n^2)$. It can mislead for small n or when constants are large: an $O(n^2)$ insertion sort with a tiny constant beats an O(n log n) algorithm with heavy overhead on arrays of 10 to 30 elements, which is why library sorts switch to insertion sort for small pieces.

Q3. Is an O(n) algorithm always faster than an $O(n^2)$ one?

No. Big-O ignores constants and describes behaviour as n grows. 1000n is O(n) and n^2 is $O(n^2)$, but n^2 is smaller for every $n < 1000$. Big-O tells you what happens eventually, so you also need to know the realistic range of n.

Q4. What is the complexity of: a loop of n iterations containing a loop of n iterations, followed by a separate loop of n iterations?

$n^2 + n$. Nested loops multiply ($n \cdot n$); sequential loops add. The dominant term is n^2, so it is $O(n^2)$.

Q5. Explain Big-O, Big-Omega and Big-Theta.

Big-O is an upper bound ("grows no faster than"), Big-Omega is a lower bound ("grows at least as fast as"), and Big-Theta is both at once (a tight bound). Linear search is O(n) and $\Omega(1)$ over all inputs, but $\Theta(n)$ in its worst case. In everyday interviews "Big-O" usually means the tight worst-case bound.

Q6. Give the best, average and worst case of quick sort. How do you avoid the worst case?

Best and average: O(n log n), because balanced partitions give $T(n) = 2T(n/2) + n$. Worst: $O(n^2)$, when the pivot is always the minimum or maximum (for example, a sorted array with a last-element pivot). Avoid it with a random pivot or median-of-three, and recurse into the smaller side first to keep the stack at O(log n).

Q7. Why is binary search O(log n)? What is the catch?

Each comparison removes half of the remaining candidates, so after k steps at most $n/2^k$ remain; this reaches zero when k exceeds $\log_2 n$. The catch: the data must be sorted (or the problem must have a monotone yes/no test). If you must sort first, the total for one search is O(n log n), which is worse than a single O(n) linear search; sorting pays off only when you search many times.

Q8. What is the time complexity of the naive recursive Fibonacci, and how do you improve it?

About $O(1.618^n)$, bounded above by $O(2^n)$, because each call spawns two more and the same subproblems are recomputed (fib(10) makes 177 calls). Memoization stores results so each fib(i) is computed once: O(n) time, O(n) space. A loop that keeps the last two values is O(n) time and O(1) space.

Q9. What is the space complexity of a recursive function?

The maximum recursion depth times the frame size, which in Big-O is the depth. Factorial has depth n: O(n). Recursive binary search has depth log n: O(log n). Naive Fibonacci has exponential time but only depth n, so its space is O(n), because calls that have returned no longer occupy the stack.

Q10. Is merge sort in place? Why choose it over quick sort?

The standard array merge sort is not in place: it needs an O(n) temporary array. Choose it when you need a guaranteed O(n log n) worst case, when stability matters (equal keys must keep their order), or for linked lists, where merging needs no extra array. Quick sort is usually faster on arrays because of cache-friendly in-place partitioning, but has an $O(n^2)$ worst case.

Q11. Why is building a heap O(n) and not O(n log n)?

Sifting down a node costs at most its height h. About $n/2^{h+1}$ nodes have height h, so the total is $\Sigma\ h \cdot n/2^{h+1} = (n/2) \cdot \Sigma\ h/2^h = (n/2) \cdot 2 = n$. Most nodes are near the leaves and move very little. (Inserting n items one by one with sift-up would be O(n log n).)

Q12. Explain amortized analysis using `ArrayList.add`.

Most adds cost O(1). When the array is full, it doubles and copies all elements. Copies happen at sizes 1, 2, 4, …, up to n, totalling less than 2n. Adding the n plain writes, n appends cost less than 3n steps, so the *average* over any sequence is at most 3 steps per add: O(1) amortized, even though a single add can cost O(n).

Q13. What is the complexity of `HashMap.get` and `HashMap.put`?

Average O(1), assuming a good hash function and a bounded load factor. The worst case is O(n) if many keys collide in one bucket; since Java 8, a long collision chain is converted to a balanced tree, making the worst case O(log n) for comparable keys.

Q14. What is the complexity of concatenating strings in a loop with `+=`? How do you fix it?

Java strings are immutable, so each `+=` copies the old string. Appending n single characters costs $1 + 2 + \ldots + n = n(n+1)/2$ copies: $O(n^2)$. Use `StringBuilder.append`, which is amortized O(1) per character, for O(n) total.

Q15. What is the complexity of this loop? `for (i = 1; i <= n; i++) for (j = 1; j <= n; j += i)`

The inner loop runs $\lceil n/i \rceil$ times, so the total is $\Sigma\lceil n/i \rceil$ over $i = 1..n \approx n(1 + \frac{1}{2} + \frac{1}{3} + \ldots + 1/n) = n \cdot H_n \approx n \ln n$. The harmonic number H_n grows like ln n. So O(n log n). (Exercise 2 in chapter 12 measures this.)

Q16. Why can no comparison-based sort beat O(n log n) in the worst case?

A comparison sort can be drawn as a binary decision tree whose leaves are the n! possible orderings. A binary tree with n! leaves has height at least $\log_2(n!)$. Since $n! \geq (n/2)^{(n/2)}$, $\log_2(n!) \geq (n/2)\ \log_2(n/2)$, which is $\Omega(n \log n)$. To beat it you must not rely only on comparisons, as counting sort and radix sort do ($O(n + k)$ when keys are small integers).

Q17. How would you find the k-th largest element, and what does each approach cost?

Sort and index: O(n log n) time. Min-heap of size k: O(n log k) time, O(k) space; good when k is small or the data streams in. Quickselect: O(n) average, $O(n^2)$ worst, in place. Mention the trade-offs and ask whether the array can be modified.

Q18. Compare three solutions to two-sum: brute force, sorting with two pointers, and a hash map.

Brute force: $O(n^2)$ time, $O(1)$ space. Sort then two pointers: $O(n \log n)$ time, $O(1)$ extra space, but you lose the original indices unless you keep them. Hash map: $O(n)$ average time, $O(n)$ space. This is a classic time-space trade-off; pick the hash map unless memory is tight.

Q19. How do you analyse a recursive function that calls itself twice on halves and does constant work?

Write $T(n) = 2T(n/2) + c$. Draw the recursion tree: level j has 2^j calls of cost c, and there are $\log_2 n + 1$ levels, so the total is $c(2n - 1)$: $O(n)$. Contrast with merge sort, where the work per call is n, making each level cost n and the total $O(n \log n)$.

Q20. The problem says $n \le 10^5$. What complexity do you aim for, and why?

A computer does about 10^8 simple operations per second. With $n = 10^5$, an $O(n^2)$ solution needs 10^{10} steps (minutes), which is too slow; $O(n \log n)$ needs about 1.7 million steps and $O(n)$ needs 100,000. So aim for $O(n \log n)$ or better: sorting, a heap, a hash map, two pointers, or binary search. If instead $n \le 20$, exponential backtracking ($2^n \approx 10^6$) is fine; if $n \le 10$, even $n!$ (3.6 million) is.

12. Ten Final Code-Analysis Exercises

For each method, (a) work out the exact value it returns for the given input by tracing by hand, (b) state the time complexity in terms of n, and (c) state the auxiliary space complexity. Write your answers before reading the solutions in chapter 15. Every method compiles and runs; the solutions were checked against the real output.

Exercise 1

```
static int ex1(int n) {
    int count = 0;
    for (int i = 0; i < n; i += 3) {
        count++;
    }
    return count;
}
```

Task: What does `ex1(10)` return? What are the time and auxiliary space complexities?

Exercise 2

```
static int ex2(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j += i) {
            count++;
        }
    }
    return count;
}
```

Task: What does `ex2(16)` return? What are the time and auxiliary space complexities?

Exercise 3

```
static int ex3(int n) {
    int count = 0;
    for (int i = n; i >= 1; i /= 2) {
        for (int j = 1; j <= n; j++) {
            count++;
        }
    }
    return count;
}
```

Task: What does `ex3(16)` return? What are the time and auxiliary space complexities?

Exercise 4

```
static int ex4(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            for (int k = 0; k < 10; k++) {
                count++;
            }
        }
    }
    return count;
}
```

Task: What does `ex4(10)` return? What are the time and auxiliary space complexities?

Exercise 5

```
static int ex5(int n) {
    if (n <= 1) return 1;
    return ex5(n / 2) + ex5(n / 2);
}
```

Task: What does `ex5(16)` return? What are the time and auxiliary space complexities?

Exercise 6

```
static int ex6(int n) {
    int count = 0;
    for (int i = 1; i <= n; i *= 2) {
        for (int j = 1; j <= i; j++) {
            count++;
        }
    }
    return count;
}
```

Task: What does `ex6(16)` return? What are the time and auxiliary space complexities?

Exercise 7

```
static int ex7(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (j > i) break;
            count++;
        }
    }
    return count;
}
```

Task: What does `ex7(10)` return? What are the time and auxiliary space complexities?

Exercise 8

```
static int ex8(int[] a, int i) {
    if (i == a.length) return 0;
    return a[i] + ex8(a, i + 1);
}
```

Task: What does `ex8(a, 0) with a = {4, 8, 15, 16, 23, 42}` return? What are the time and auxiliary space complexities?

Exercise 9

```
static int ex9(int n) {
    int count = 0;
    for (int i = 1; i < n; i *= 2) {
        for (int j = n; j > 0; j /= 2) {
            count++;
        }
    }
    return count;
}
```

Task: What does `ex9(16)` return? What are the time and auxiliary space complexities?

Exercise 10

```java
static int ex10(int n) {
    int count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j *= 2) {
            for (int k = 0; k < j; k++) {
                count++;
            }
        }
    }
    return count;
}
```

Task: What does `ex10(16)` return? What are the time and auxiliary space complexities?

13. Cheat Sheet

13.1 The growth ladder

Table 13.1

Class	Name	Typical code or algorithm	n = 1,000 → steps
O(1)	constant	array index, swap, hash lookup (avg)	1
O(log n)	logarithmic	binary search, halving/doubling loop, heap push/pop	≈ 10
O(n)	linear	single pass, linear search, sum, merge step	1,000
O(n log n)	linearithmic	merge sort, heap sort, quick sort (avg), sort then scan	≈ 10,000
$O(n^2)$	quadratic	nested loops, bubble/selection/insertion sort	1,000,000
$O(n^3)$	cubic	three nested loops, naive matrix multiply	10^9
$O(2^n)$	exponential	all subsets, naive Fibonacci	≈ 10^{301}
O(n!)	factorial	all permutations, brute-force travelling salesman	astronomical

13.2 Loop patterns

Table 13.2

Pattern	Complexity	Why
one loop to n; `i += c`	O(n)	n/c iterations
loop with `i *= 2` or `i /= 2`	O(log n)	halves or doubles the range
two loops in sequence	add → larger one	n + m
loop inside loop (independent)	multiply	n · m
inner `j < i`	$O(n^2)$	n(n−1)/2
inner `j *= 2` inside `i < n`	O(n log n)	n · log n
outer `i /= 2`, inner `j < i`	O(n)	n + n/2 + … < 2n
inner `j += i`	O(n log n)	$\Sigma\, n/i = n \cdot H_n$
inner runs i^2 times	$O(n^3)$	$\Sigma\, i^2 = n(n+1)(2n+1)/6$
constant inner loop	O(n)	5n, 100n, …
recursion with 1 call on n−1	O(n)	chain of n calls
recursion with 2 calls on n−1	$O(2^n)$	binary tree of depth n
recursion with 2 calls on n/2 + O(n) work	O(n log n)	merge sort
recursion with 2 calls on n/2 + O(1) work	O(n)	2n − 1 calls

13.3 Searching and sorting

Table 13.3

Algorithm	Best	Average	Worst	Space	Stable
Linear search	O(1)	O(n)	O(n)	O(1)	—
Binary search (sorted)	O(1)	O(log n)	O(log n)	O(1) / O(log n) rec.	—
Bubble (flag)	O(n)	O(n²)	O(n²)	O(1)	yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	no
Insertion	O(n)	O(n²)	O(n²)	O(1)	yes
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	yes
Quick (last pivot)	O(n log n)	O(n log n)	O(n²)	O(log n) avg	no
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	no

13.4 Java collections

Table 13.4

Structure	Access / search	Insert / delete	Notes
`int[]`	get/set O(1); search O(n)	O(n) (shift)	fixed size
`ArrayList`	`get` O(1); `contains` O(n)	add at end O(1) amortized; add/remove in the middle or at index 0: O(n)	backing array doubles
`LinkedList`	`get(i)` O(n)	at the ends O(1)	poor cache behaviour
`ArrayDeque`	—	both ends O(1)	use as stack or queue
`HashMap` / `HashSet`	average O(1)	average O(1)	worst O(n); O(log n) with tree bins
`TreeMap` / `TreeSet`	O(log n)	O(log n)	sorted order
`PriorityQueue`	`peek` O(1)	`offer`, `poll` O(log n)	`remove(Object)` is O(n)
`StringBuilder`	—	`append` amortized O(1)	prefer over `String +=`

13.5 Sums and series

Table 13.5

Series	Value
1 + 2 + … + n	n(n+1)/2
1² + 2² + … + n²	n(n+1)(2n+1)/6
1³ + 2³ + … + n³	[n(n+1)/2]²
1 + 2 + 4 + … + 2^k	$2^{k+1} - 1$
n + n/2 + n/4 + …	< 2n
1 + ½ + ⅓ + … + 1/n	≈ ln n (harmonic number)
$\log_2(n!)$	Θ(n log n)

13.6 What n can I afford?

Table 13.6 (about 10^8 simple operations per second)

Constraint	Aim for
$n \le 10$	O(n!) is fine
$n \le 20 – 25$	$O(2^n)$
$n \le 500$	$O(n^3)$
$n \le 5{,}000$	$O(n^2)$
$n \le 10^6$	O(n log n)
$n \le 10^8$	O(n)
larger	O(log n) or O(1)

14. One-Page Revision Sheet

Golden rules

Rule	Meaning
Drop constants	5n, n/2, n+100 → O(n)
Keep dominant term	$n^2 + n \log n + 7 \to O(n^2)$
Sequential adds, nested multiplies	$n + n^2 \to O(n^2)$; $n \cdot \log n$
Different inputs, different letters	O(a + b), O(a · b)
Log base irrelevant	$\log_2 n = c \cdot \log_{10} n$

The eight classes

Class	Example	Analogy
O(1)	index, swap	top plate
O(log n)	binary search	halve a range
O(n)	one pass	read the list
O(n log n)	merge / heap sort	split + merge
$O(n^2)$	all pairs	handshakes
$O(n^3)$	3 loops, matmul	trios
$O(2^n)$	subsets, fib	pack / skip
O(n!)	permutations	seat orders

Loops

Loop shape	Count
`i += c`	n/c → O(n)
`i *= 2` or `i /= 2`	$\log_2 n$
`j < i` in `i < n`	$n(n-1)/2 \to n^2$
`j *= 2` in `i < n`	n log n
outer `i /= 2`, inner `j < i`	< 2n → O(n)
`j += i` in `i ≤ n`	$n \cdot H_n \to n \log n$
`j ≤ i*i`	$\Sigma i^2 \approx n^3/3$
three nested	n^3
early break	worst case decides

Space

Space	Rule
Auxiliary	extra memory, not the input
Recursion	stack depth: n, log n, …
In place	O(1) extra: bubble, heap, reverse
Not in place	merge sort O(n); `new int[n]`

Sorting (lg = $\log_2$)

Sort	Best	Avg	Worst	Space	Stable
Bubble	n	n^2	n^2	1	yes
Selection	n^2	n^2	n^2	1	no
Insertion	n	n^2	n^2	1	yes
Merge	n lg n	n lg n	n lg n	n	yes
Quick	n lg n	n lg n	n^2	lg n	no
Heap	n lg n	n lg n	n lg n	1	no

Searching

Search	Best	Worst	Space
Linear	1	n	1
Binary (sorted)	1	log n	1 (loop) / log n (rec.)

Recurrences

Recurrence	Result	Example
T(n−1) + c	O(n)	factorial
T(n/2) + c	O(log n)	binary search
2T(n/2) + n	O(n log n)	merge sort
2T(n/2) + c	O(n)	tree walk
T(n−1) + n	$O(n^2)$	worst quick sort
2T(n−1) + c	$O(2^n)$	naive fib

Java costs

Collection	Key costs
`ArrayList`	get O(1); add end O(1) am.; middle O(n)
`HashMap`	get/put avg O(1)
`TreeMap`	O(log n)
`PriorityQueue`	offer/poll O(log n); peek O(1)
`String +=` loop	$O(n^2)$; use `StringBuilder`

Sums: $1+\ldots+n = n(n+1)/2$; $\Sigma k^2 = n(n+1)(2n+1)/6$; $1+2+\ldots+2^k = 2^{k+1}-1$; $n + n/2 + \ldots < 2n$.

Limits (10^8 ops/s): n ≤ 10: n!; 25: 2^n; 500: n^3; 5,000: n^2; 10^6: n log n; 10^8: n.

Interview script: state n → count the innermost statement → nested multiplies, sequential adds → keep the dominant term → best vs worst → auxiliary space incl. recursion depth → mention the time–space trade-off.

15. Answers and Explanations

15.1 MCQ answer key

Beginner level

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
B	D	A	C	B	A	D	C	A	B

Intermediate level

Q11	Q12	Q13	Q14	Q15	Q16	Q17	Q18	Q19	Q20
C	A	D	B	C	D	A	B	D	C

Interview level

Q21	Q22	Q23	Q24	Q25	Q26	Q27	Q28	Q29	Q30
A	C	B	D	A	B	C	D	B	A

Answer letters are spread out on purpose (A: 8, B: 8, C: 7, D: 7); do not guess by pattern.

15.2 MCQ explanations

Q1. Answer B: O(1). Array indexing computes the memory address directly from the start address and the index, so the cost does not depend on n: O(1).

Q2. Answer D: O(n²). Keep only the fastest-growing term, $4n^2$, then drop its constant factor 4. "O($4n^2$)" is not simplified.

Q3. Answer A: O(n). With `i += 2` the loop runs ⌈n/2⌉ times. A constant factor of ½ does not change the class, so it is O(n). (For n = 10 it runs 5 times.)

Q4. Answer C: 6. i takes the values 1, 2, 4, 8, 16, 32 and the loop stops when i reaches 64. That is 6 iterations = $\log_2 64$. Checking the neighbours: 5 would stop one value early; 7 would include 64 itself, but $64 < 64$ is false.

Q5. Answer B: O(n). Sequential loops add: n + n = 2n, and 2n simplifies to O(n). They would multiply only if one were inside the other.

Q6. Answer A: 100. The inner loop runs 10 times for each of the 10 outer iterations: 10 × 10 = 100.

Q7. Answer D: 2^n. Exponential growth overtakes every polynomial eventually. For n = 100: $n^3 = 10^6$ but $2^n \approx 1.27 \times 10^{30}$.

Q8. Answer C: O(n). In the worst case the target is last or absent, so all n elements are compared.

Q9. Answer A: O(1). Only a fixed number of variables (`sum`, the loop index) are used. The input array is not counted as auxiliary space.

Q10. Answer B: The array is sorted. Binary search discards half the array by comparing with the middle element, which is valid only if the elements are in order. Duplicates, length and sign are irrelevant.

Q11. Answer C: 45. The inner loop runs i times for each i = 0..9: 0 + 1 + ... + 9 = 9·10/2 = 45. (55 would be for `j <= i`; 100 is for a full square.)

Q12. Answer A: 64. The outer loop runs 16 times. The inner loop j = 1, 2, 4, 8 runs 4 times. 16 × 4 = 64.

Q13. Answer D: O(n). The inner loop runs n, n/2, n/4, … times. This geometric series sums to less than 2n, so the total is O(n) (for n = 16 the count is 31).

Q14. Answer B: O(n log n). $\log_2 n$ levels, each costing n in total (Unrolling 6.3). This is merge sort.

Q15. Answer C: O(n). The deepest point has n frames alive at once (fact(n), fact(n−1), …, fact(1)).

Q16. Answer D: O(n). On sorted input each element is compared once with its left neighbour and stays in place: n − 1 comparisons.

Q17. Answer A: Merge sort. Merge sort always splits evenly and merges linearly. Quick sort degrades to $O(n^2)$ on sorted input with a last-element pivot; bubble (with flag) and insertion are $O(n)$ on sorted input and $O(n^2)$ on reversed input.

Q18. Answer B: 28. Pass i makes $n - 1 - i$ comparisons: $7 + 6 + 5 + 4 + 3 + 2 + 1 = 28 = 8 \cdot 7/2$, whatever the data.

Q19. Answer D: O(n). The inner loop is always exactly 5 iterations, so the total is $5n = O(n)$.

Q20. Answer C: O(n) time and O(n) space. n insertions at O(1) average each, and the set may hold n elements. The nested-loop version is $O(n^2)$ time but O(1) space.

Q21. Answer A: 177. $C(n) = 1 + C(n-1) + C(n-2)$ with $C(0) = C(1) = 1$ gives $C(10) = 177$ (measured by running the code). 55 and 89 are Fibonacci numbers, not call counts; 1,024 is 2^{10}, an upper-bound style figure.

Q22. Answer C: O(n log n). For a given i the inner loop runs $\lceil n/i \rceil$ times. The total $\Sigma \lceil n/i \rceil$ for i = 1..n is about $n \cdot H_n \approx n \ln n$ (harmonic series).

Q23. Answer B: O(n⁴). For each i the k-loop runs $\Sigma(i \cdot m)$ for m = 1..i = $i^2(i+1)/2$ times; summing over i gives about $n^4/8$.

Q24. Answer D: O(m + n). Each output element is produced by one comparison or copy, and there are m + n of them. Re-sorting would cost $O((m+n) \log(m+n))$.

Q25. Answer A: O(n²). Each partition removes only the pivot: $T(n) = T(n-1) + n - 1 = n(n-1)/2$ comparisons (measured: 4,950 for n = 100).

Q26. Answer B: O(n). Merging needs a temporary array of up to n elements. The recursion adds O(log n) frames, which is dominated by O(n).

Q27. Answer C: O(n). Most nodes are near the bottom and sift only a short way. The sum $\Sigma\, h \cdot n/2^{h+1}$ is at most n (Derivation 8.6).

Q28. Answer D: O(n). $T(n) = 2T(n/2) + c$. Level j has 2^j calls; the sum $1 + 2 + \ldots + n$ is about 2n: O(n) (Unrolling 6.6). The Master Theorem case $a > b^d$ agrees.

Q29. Answer B: O(1). Occasionally the backing array doubles and all elements are copied, but copying happens at sizes 1, 2, 4, …, whose total is less than 2n. So n appends cost less than 3n steps, i.e. O(1) amortized each.

Q30. Answer A: O(n²). Each `+=` creates a new string and copies the whole old one: $1 + 2 + \ldots + n = n(n+1)/2$ character copies. `StringBuilder` makes it O(n).

15.3 Solutions to the ten code-analysis exercises

Values marked "measured" come from running the exact Java code of chapter 12.

Exercise 1. `ex1(10)` returns **4** (measured).

i takes the values 0, 3, 6, 9 (the next value, 12, fails `i < 10`). In general the loop runs $\lceil n/3 \rceil$ times (Derivation 4.1).

Time: O(n). Auxiliary space: O(1).

Exercise 2. `ex2(16)` returns **61** (measured).

For a fixed i, j takes the values 1, 1+i, 1+2i, … while $j \le n$, which is $\lceil n/i \rceil$ values. For n = 16 and i = 1..16 these are 16, 8, 6, 4, 4, 3, 3, 2, 2, 2, 2, 2, 2, 2, 2, 1, which sum to 61. In general $\Sigma \lceil n/i \rceil \approx n \cdot H_n \approx n \ln n$.

Time: O(n log n). Auxiliary space: O(1).

Exercise 3. `ex3(16)` returns **80** (measured).

The outer loop sets i = 16, 8, 4, 2, 1: that is 5 values ($\lfloor \log_2 16 \rfloor + 1$). The inner loop always runs n = 16 times (its bound does not depend on i). $5 \times 16 = 80$. In general $n(\lfloor \log_2 n \rfloor + 1)$.

Time: O(n log n). Auxiliary space: O(1).

Exercise 4. `ex4(10)` returns **550** (measured).

For each i the middle loop runs n − i times, so the two outer loops execute Σ for i = 0..9 of (10 − i) = 10 + 9 + … + 1 = 55 times. The innermost loop adds a factor of 10: 55 × 10 = 550. In general 10 · n(n+1)/2.

Time: O(n²). Auxiliary space: O(1).

Exercise 5. `ex5(16)` returns **16** (measured).

The method returns 1 at each base case and adds the two results, so it returns the number of leaves of the call tree. For n = 16 the argument halves 16 → 8 → 4 → 2 → 1: a full binary tree with 16 leaves and 31 calls in total (2n − 1). T(n) = 2T(n/2) + c gives O(n) (Unrolling 6.6).

Time: O(n). Auxiliary space: O(log n) — the recursion depth is ⌊log₂ n⌋ + 1 = 5 frames.

Exercise 6. `ex6(16)` returns **31** (measured).

i = 1, 2, 4, 8, 16 and the inner loop runs i times: 1 + 2 + 4 + 8 + 16 = 31 = $2^5 - 1$. This is a geometric series with sum less than 2n (Derivation 4.5), so the "log n outer × up to n inner" shape is only O(n).

Time: O(n). Auxiliary space: O(1).

Exercise 7. `ex7(10)` returns **55** (measured).

The inner loop breaks as soon as j > i, so for each i it executes the body for j = 0..i, which is i + 1 times (it stops after testing j = i + 1). Total = 1 + 2 + … + 10 = 55 = n(n+1)/2. The break turns the full square into a triangle: still O(n²).

Time: O(n²). Auxiliary space: O(1).

Exercise 8. `ex8(a, 0)` returns **108** (measured).

4 + 8 + 15 + 16 + 23 + 42 = 108. The recursion makes one call per element plus one for the base case i = n: 7 calls for n = 6, all alive at the deepest point.

Time: O(n). Auxiliary space: O(n) — stack depth n + 1.

Exercise 9. `ex9(16)` returns **20** (measured).

The outer loop (i = 1, 2, 4, 8) runs 4 times = ⌈log₂ 16⌉. The inner loop (j = 16, 8, 4, 2, 1) runs 5 times = ⌊log₂ 16⌋ + 1, independently of i. 4 × 5 = 20. In general about (log n)².

Time: O(log² n). Auxiliary space: O(1).

Exercise 10. `ex10(16)` returns **496** (measured).

For a fixed i, the j and k loops execute Σ j for j = 1, 2, 4, 8, 16 = 31 times (the k-loop runs j times). The outer loop repeats this 16 times: 16 × 31 = 496. The pair of inner loops costs 2^(⌊log₂ n⌋ + 1) − 1 < 2n, so the total is below 2n².

Time: O(n²). Auxiliary space: O(1).

15.4 How this guide was verified

- **Loop and recursion counts:** 252 measured values from `Complexity.java` (OpenJDK 21) compared with closed-form formulas; no mismatches.
- **Exercises:** 136 measured values from `Exercises.java` compared with the formulas used in the solutions; no mismatches.
- **Sorting and searching:** all six sorts were compared with `Arrays.sort` on 3,000 random arrays each; linear and binary search (iterative and recursive) on 3,000 random sorted arrays.
- **Problems:** each brute-force and optimised pair was run on the documented examples and compared on thousands of random inputs (5,000 rounds for the array problems, 5,000 for strings, 2,000 for missing-number, peak and matrix problems; square root for every x up to 5,000 and for the largest `int` values; stair climbing for n = 0..25).

- **Binary search bound:** the worst case $\lfloor \log_2 n \rfloor + 1$ was confirmed for every array size from 1 to 299 and every target.
- **Traces and tables:** generated by Python re-implementations of the same algorithms and asserted equal to the Java results.
- All listings were compiled and run as shown, with spacing only adjusted for the page.

Appendix: Maths Refresher

Logarithms

- $\log_2 x = y$ means $2^y = x$. $\log_2 8 = 3$, $\log_2 1024 = 10$.
- $\log(ab) = \log a + \log b$; $\log(a^b) = b \cdot \log a$; $\log_a x = \log_b x / \log_b a$ (change of base, only a constant factor).
- $\lfloor \log_2 n \rfloor + 1$ is the number of binary digits of n; it is also the number of times you can halve n (with integer division) before reaching 0.

Geometric series

Derivation A.1. For ratio $r \neq 1$: $1 + r + r^2 + \ldots + r^m = (r^{m+1} - 1)/(r - 1)$. **Proof:** let S be the sum. Then $rS = r + r^2 + \ldots + r^{m+1}$. Subtract: $rS - S = r^{m+1} - 1$, so $S = (r^{m+1} - 1)/(r - 1)$. With $r = 2$ this gives $1 + 2 + 4 + \ldots + 2^m = 2^{m+1} - 1$. With $r = ½$ and infinitely many terms, $S = 1/(1 - ½) = 2$.

Harmonic numbers

$H_n = 1 + ½ + ⅓ + \ldots + 1/n$. Comparing each term $1/k$ with the area under the curve $1/x$ between k and $k + 1$ shows $H_n \geq \ln(n + 1)$, and comparing with the area between $k - 1$ and k shows $H_n \leq 1 + \ln n$. So $H_n = \Theta(\log n)$. Example: $H_{100} \approx 5.19$ and $\ln 100 \approx 4.61$.

Factorials

$n! = n \cdot (n-1) \cdot \ldots \cdot 1$. Stirling's approximation: $n! \approx \sqrt{(2\pi n)} \cdot (n/e)^n$, so $\log_2(n!) = n \log_2 n - 1.44n + O(\log n) = \Theta(n \log n)$. A simple bound: the top half of the factors are each at least $n/2$, so $n! \geq (n/2)^{(n/2)}$.

Final note. Practice questions in this guide were written for study purposes. They are not claimed to be actual Google, Microsoft or LeetCode questions. Good luck with your internship interviews.